

WEB APPLICATION AND APIS FOR VIDEO DESCRIPTION

A PROJECT REPORT
SUBMITTED TO THE DEPARTMENT OF COMPUTER SCIENCE
AND THE COMMITTEE ON GRADUATE STUDIES OF
SAN FRANCISCO STATE UNIVERSITY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
MASTER OF SCIENCE IN COMPUTER SCIENCE

BY
VAISHALI BISHT
MAY 2020

ABSTRACT

The project aims to increase video accessibility for blind and visually impaired users. Video description, audio narration generated for scenes that are played along with the video soundtrack, often requires professional expertise and consumes a lot of time. We developed a system that helps amateur sighted volunteers in generating descriptions of the video by utilizing computer vision and image processing techniques such as automatic scene segmentation, description generation, and capturing text-on-screen. A framework is developed to automate scene segmentation and description generation process by integrating the system with publicly available computer vision APIs. The automation of video description helps in reducing the cognitive load of the describers which in turn reduces the turnaround time and cost of the video description. It also helps in increasing the description quality. To study the impact of the new system on describers as well as blind users, a user study has been performed. The novice describers found the new system significantly easier to use and helped in creating video descriptions faster as per the User study conducted. The results from the user study with the blind and visually impaired users showed that the description quality of AI-supported video was better than a video with no AI support. This report provides details about backend APIs development, user study system development, the current stage of the project, and future directions.

I certify that the Abstract is a correct representation of the content of this project.

Chair, Committee Member

Date

TABLE OF CONTENTS

1. Introduction	5
1.1 Motivation	6
1.2 Related Work	7
2. Application Design	10
2.1 Application Overview	10
2.2 Application Flow Diagram	10
2.3 Application Architecture	12
2.4 Technical Stack	19
2.5 Contribution to the Project and Challenges	20
3. Implementation	22
3.1 Automation of Scene segmentation	22
3.1.2 Sequence Diagram	22
3.1.3 Upload Video feature Flow Diagram	23
3.1.4 Insert Video Data into Database Flow Diagram	24
3.1.5 Back End API	26
3.2 Merge Scene based on Audio Conflicts	32
3.3 OCR Cleanup	34
3.4 Integration Of Pythia Captioning Service	37
3.5 Filter Description using BLEU Score	41
3.6 Database Schema	49
4. User Study Design And Development	55
4.1 User Study I - Phase I	55

4.1.1 User Study Preparation	55
4.1.2 Experimental Design	56
4.1.3 User Interface	57
4.1.3.1 Home Page	57
4.1.3.2 Video List page	58
4.1.4 Backend APIs	59
4.2 User Study I - Phase II	60
4.2.1 Experimental Design	60
4.2.2 Video Rating Page	61
4.2.3 Backend APIs	63
4.3 User Study II	63
4.3.1 Experiment setup	64
4.3.2 Video Page With AI	65
4.3.3 Home page	71
4.3.4 Prep Session	71
4.3.4.1 Prep Session - Participant page	72
4.3.4.2 Prep Session – Admin page	72
4.3.5 Session III	73
4.3.5.1 Participant page – Session III	73
4.3.5.2 Admin page - Session III	74
4.3.6 Backend APIs	75
5. Conclusion	76
6. Future Work	76
7. References	78

1. INTRODUCTION

Online videos have become an important resource in many aspects of our life which includes education, employment, recreation, and more. Therefore it is important that online videos should be accessible to everyone in order to provide equal access and equal opportunity to people with disabilities [1]. However, the vast majority of video material on the Internet is currently not accessible to the millions of visually impaired people, thus creating an increasing digital divide between blind and sighted users [2].

Video description makes the video content like online videos, tv programs, movies accessible to individuals who are blind or visually impaired. Video Description is defined as “the narration added to the soundtrack to describe important visual details that cannot be understood from the main soundtrack alone” [3]. Traditional video description requires human labor for the entire workflow. Once the description transcript is written, human voice actors are hired to record the description. This method is complex, costly, and quite time-consuming [4].

Computer vision is widely used to analyze video content like object detection, face detection, scene changes. We designed a system by amalgamating human, computer vision, and image processing techniques to help amateur describer to produce high-quality video descriptions. Our system provides an interface that contains automated scene segmentation, scene captions, and text on the screen. The machine-generated descriptions are not of good quality yet, therefore a human is required in the workflow to edit the descriptions, thereby promoting an alliance between Humans and AI. These descriptions are further converted from text to speech and played along with the video soundtrack.

This project focuses on surveying possibly usable vision and image processing tools and creating a user study system to evaluate which interfaces or features are effective.

1.1 Motivation

There has been a huge growth in online video over the years. Individuals who are visually impaired, are constantly being excluded from online video because accessibility is lacking tremendously. As online video increases even more quickly the gap between accessible and non-accessible video content is growing exponentially [5].

Video description plays a crucial role to provide video accessibility to blind and visually impaired users. Video description requires professional expertise, a lot of time, and significant cost. The video description cost and effort varies from video to video. Professional video description can cost up to hundreds or thousands of dollars [2]. There are free tools like LiveDescribe [6] that allow amateur describers to create audio descriptions. Another free website is YouDescribe [7] which is a web-based platform that enables anyone to add an audio description to the videos uploaded on Youtube. However, the describers find it very difficult to describe the videos due to constraints like lack of training, limited vocabulary, or writing skills. Also, the description quality of the volunteers varies a lot.

The process of audio description can be considered somewhat of an art. Understanding when to describe, what to describe, and when not to describe can feel complex, and finding the right balance takes time for the novice describers [8]. Also, the descriptions need to be concise as they will be played during the pause in the dialogue, or sometimes the video will be paused to insert descriptions.

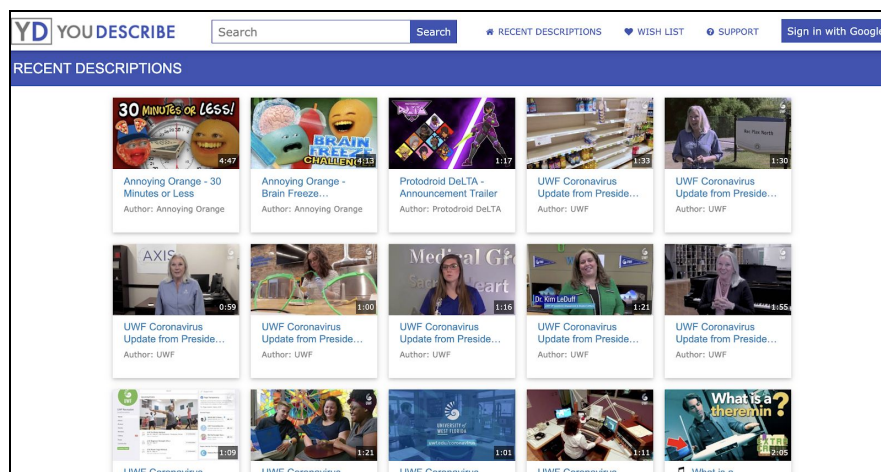


Figure 1. YouDescribe website

There is a need for a system that can simplify the video description process. The system should provide assistance to describers like automating scene segmentation, scene descriptions, etc. The scene description can be helpful in guiding the describers of how much detail they should provide [2]. Using AI components in video description can help in reducing the cognitive workload of the describers thereby encouraging people to volunteer for video descriptions.

1.2 Related Work

In this section, we will discuss existing tools and the work that has been done in the domain of audio description to provide video accessibility.

LiveDescribe

LiveDescribe is a stand-alone application that allows amateur audio describers to create and edit audio descriptions for blind audiences [6]. LiveDescribe is developed by the Inclusive Media and Design Center at Canada's Ryerson University. The prototype of LiveDescribe was developed in 2006. LiveDescribe enables the production of real-time video descriptions for online broadcasts. It includes features such as two-button operation and automatic detection of silent periods [9]. Figure 2 shows the user interface of LiveDescribe.

Features Comparison with our system:

- LiveDescribe detects the gaps where the audio can be inserted in the video without interrupting the original audio track of the video. The app uses graphs to show the information of the position where the audio can be inserted. The audio descriptions are inserted in between the dialogue. LiveDescribe allows the users to upload the audio descriptions to the website and share it with the community [6]. On the other hand, our application relies on scene segmentation and description of scenes in text which is further converted to speech and played along with the video. Our application allows the user to control the scene generated. The user can merge similar scenes. The application identifies

the audio conflicts in the video, like where dialogue is interrupted in a scene and provides recommendations to the user to merge the scenes.

- LiveDescribe application has a feature where a user can create a real-time video description for online broadcasts. This feature imposed a high cognitive load on the describers [9]. Our application does not support real-time video description as it takes time to process the video and generate scene segmentation, captions and OCR. A user study conducted proved our system to be less mentally demanding.

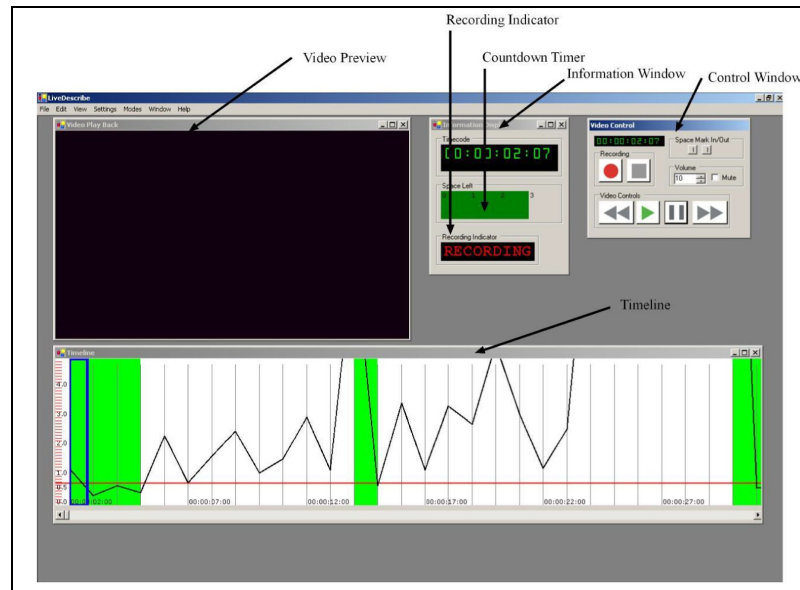


Figure 2. LiveDescribe user interface

3PlayMedia

3PlayMedia [10] provides closed captioning, transcription, audio description, and video translation services to make video accessibility easy. The company is based in Boston, MA, and has been operating since 2007. The audio description service has features like - audio description editor, synthesized speech, extended audio description, and 3Play Plugin [5].

- The audio description editor provides a user interface where a user can edit the descriptions. The user can also play the descriptions using synthesized speech. The synthesized speech has eighteen English voice options, three Spanish voice options, and

three-speed options [5]. Our application also has a user interface where a user is provided with scene segmentation and descriptions. The user can edit the descriptions. We also make use of synthesized speech for playing the descriptions. Currently, we support only one English voice option.

- 3Play Plugin enables users to add captions, interactive transcripts, audio descriptions to the video players. The plugin is an iframe embed code that plays the secondary audio description track simultaneously along with the video. The plugin is compatible with most video players and publishing platforms [11].

Features of the Plugin:

1. Interactive Transcript and Video Search: It allows the user to search for a term where that word is spoken in the video.
 2. A user can turn on and off the audio description in the video by clicking on the “AD” icon.
 3. A user can look for the relevant keywords in the transcript by clicking the “magic wand” icon.
 4. A pdf of the video transcript can be downloaded by clicking the “download” icon.
- 3Play Media’s audio description services are paid. The service starts from \$9 per minute of video length. The system we have developed will be integrated with the YouDescribe website, which is a free platform for creating video descriptions.

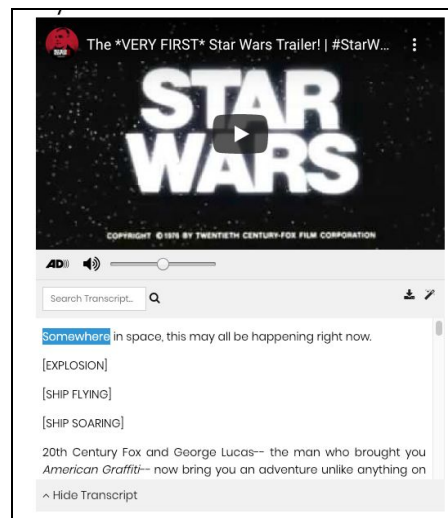


Figure 3. 3Play plugin

2. APPLICATION DESIGN

In this section, we will discuss the design of the application in detail. A high level overview of the features of the application and how different objects of the application interact with each other is discussed in section 2.1.

2.1 Application Overview

The application provides a video description interface to assist users in producing video description. The interface consists of scene segmentation, scene description, and text on screen. The user can edit the scene description and then the description is further converted to speech. The descriptions are played along with the video using synthesized speech. The different stages of video description are discussed in detail in section 4.2.

2.2 Application Flow Diagram

The workflow of the application when the user provides the youtube URL of the video is shown in figure 4 and described below.

1. **Input video URL:** URL of the YouTube video for which descriptions have to be generated.
2. **Video processing:** The video is downloaded from YouTube using the URL. Further video processing is divided into two parts:
 - a. **Scene segmentation:** The app uploads the downloaded video file to Video Indexer using REST API. Video Indexer takes some time to process the video depending on the video length. It takes around 5-10 minutes to process a video of the length of 5 minutes. After analyzing the video, Video Indexer will notify the application using a callback URL. When the application receives a notification, video insight is fetched from Video Indexer. The video insight contains the scene segmentation, keyframes, and OCR. All this information is parsed and saved to the database.

- b. **Description generation:** The video insight contains keyframes in a scene for a video. The keyframes metadata contains image URLs that are publically accessible. These keyframes are sent to the Azure Computer Vision API [13]. The API returns the captions for the keyframes. The captions are aggregated to form the description for a scene. The keyframes are also sent to the IBM Watson Max Scene Classifier API [14]. The API detects the environment of each keyframe. The data from Computer Vision API and IBM API are saved in the database. Currently, the environment data is not used by the application but stored in the database for the future.

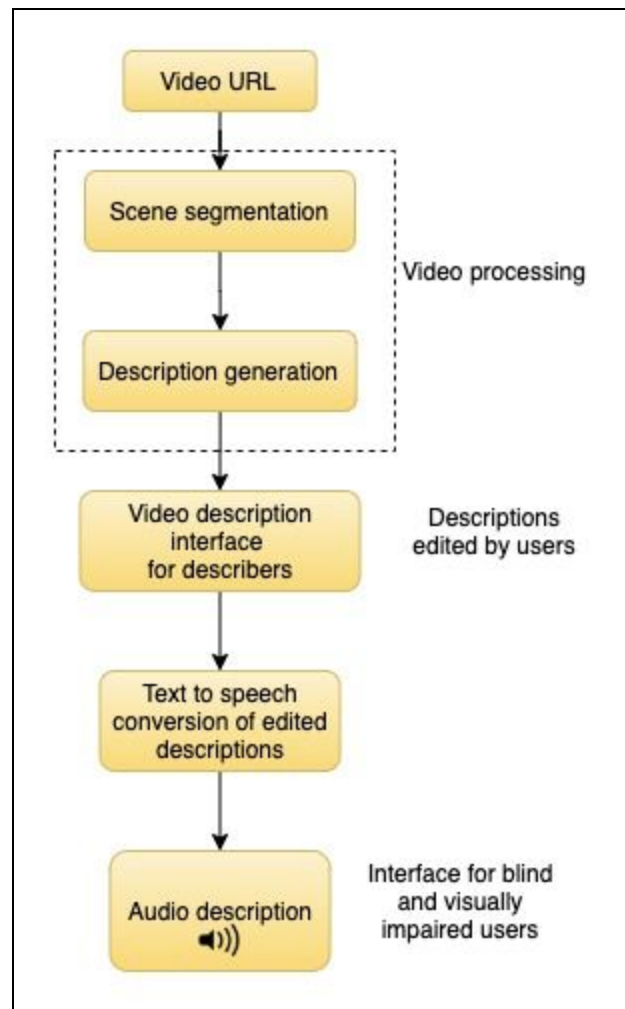


Figure 4. Application flow diagram

3. **Video description user interface:** The scene segmentation and generated descriptions are provided to the user for editing. The edited descriptions are stored in the database.
4. **Text to speech conversion:** The edited descriptions are then sent to Google Cloud Text To Speech API [15] which produces audio files in .mp3 format for all the descriptions. These audio files are stored in the project's directory.
5. **Playing audio description:** A user interface is created for blind and visually impaired users where the audio files are played with the video original soundtrack. The audio description is played in the gaps between the dialogues. If the audio is too long for a scene, the video is paused and the audio description is played.

2.3 Application Architecture

Figure 5 shows the architecture of the application. The application is developed using Express.js as the web development framework and Node.js as the runtime environment. The application depends on publicly available APIs such as Microsoft Video Indexer, Azure Computer Vision, Google Text to Speech services, etc. These APIs provide capabilities like scene detection, caption generation, text to speech conversion, etc. The various components of the application are discussed in detail below.

2.3.1 Web Application

The system is an Express NodeJs web application. The app uses a Model-View-Controller pattern. The Frontend is created using HTML, CSS, Javascript, and JQuery. Express is used as the web framework for building the app and Node as the runtime environment. The app is hosted on AWS EC2 instance.

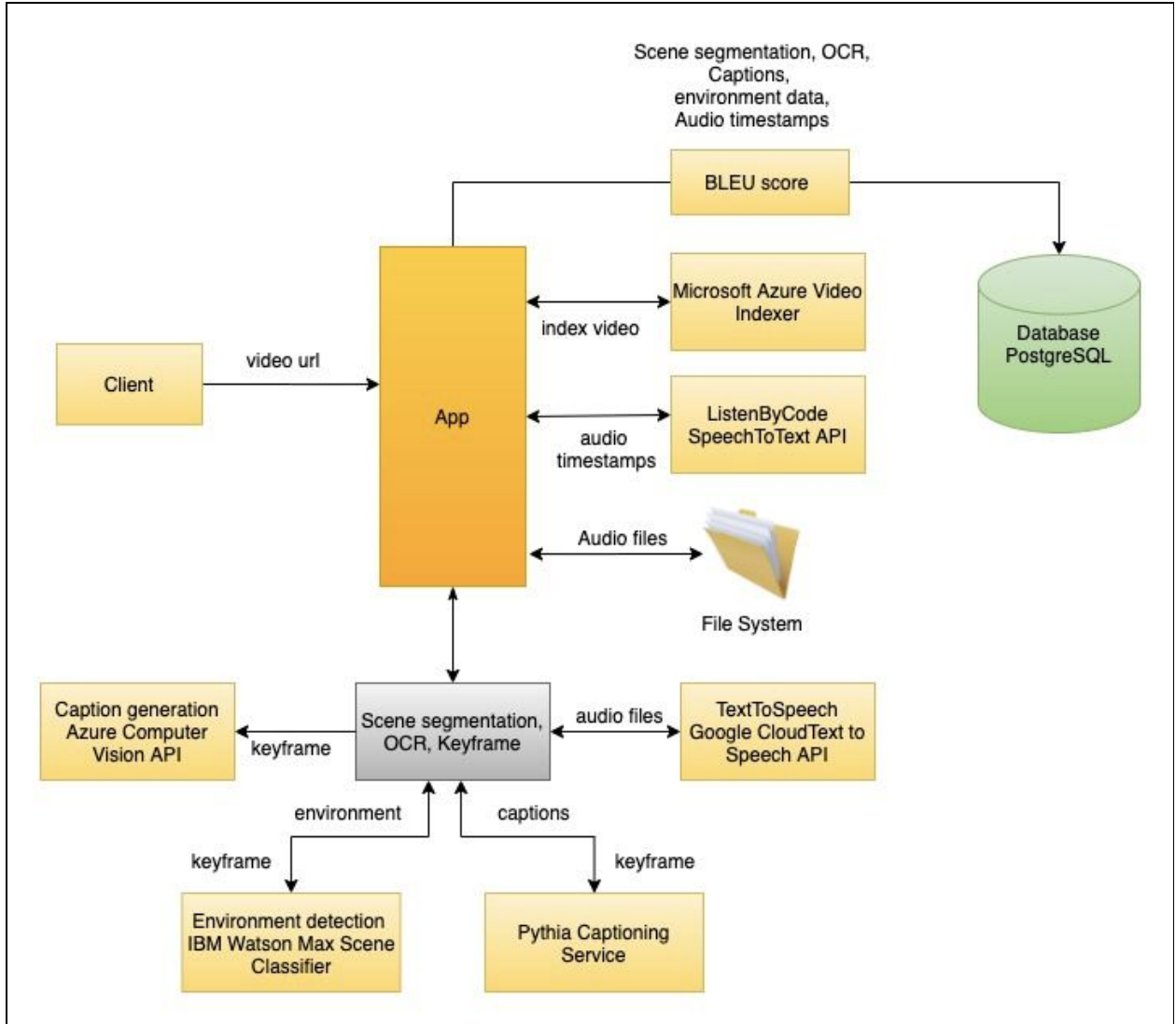


Figure 5. Application architecture

2.3.2 Microsoft Video Indexer

Microsoft Video Indexer [12] analyzes a video both on visuals and audio by using different machine learning models. It extracts a deep insight from a video based on voice, vocals, and visuals. Video Indexer can retrieve video insights like scene segmentation, shot detection, keyframe detection, face detection, celebrity identification, visual text recognition (OCR), label identification, etc. Apart from video insight, it also extracts audio insights like audio transcription, automatic language detection, closed captioning, speaker enumeration, audio effects, etc. Video Indexer provides a good set of APIs that can be easily integrated with the

application. Other alternatives for video analysis are Google Vision [16], Amazon Rekognition [17].

The application uploads the video to Video Indexer for processing. Once the video is processed, the application fetches the video insight using Video Indexer API. Figure 6 shows the response JSON containing video insight from Video Indexer. The insights include scenes, shots, transcripts, OCRs, faces, topics, blocks, etc. Each insight type includes instances of time ranges that show when the insight appears in the video. The application currently makes use of the scene segmentation, keyframes, and OCR. Scene segmentation is explained in detail in section 2.3.3.

```
{
  "partition": null,
  "description": null,
  "privacyMode": "Public",
  "state": "Processed",
  "accountId": "5a8c6ddf-900f-48b4-8a3a-0cd5cf48cb4b",
  "id": "f3f76a2411",
  "name": "Cheer On Demand Advertisement",
  "userName": "Umang Mathur",
  "created": "2020-05-18T02:43:16.7865071+00:00",
  "isOwned": true,
  "isEditable": true,
  "isBase": true,
  "durationInSeconds": 18,
  "summarizedInsights": {[]},
  "videos": [],
  "videosRanges": []
}
```

Figure 6. JSON response from video indexer

```

"videos": [
  {
    "accountId": "5a8c6ddf-900f-48b4-8a3a-0cd5cf48cb4b",
    "id": "f3f76a2411",
    "state": "Processed",
    "moderationState": "OK",
    "reviewState": "None",
    "privacyMode": "Public",
    "processingProgress": "100%",
    "failureCode": "None",
    "failureMessage": "",
    "externalId": null,
    "externalUrl": null,
    "metadata": null,
    "insights": {
      "version": "1.0.0.0",
      "duration": "0:00:18.454",
      "sourceLanguage": "en-US",
      "sourceLanguages": [{}],
      "language": "en-US",
      "languages": [{}],
      "ocr": [{}],
      "keywords": [{}],
      "labels": [{}],
      "scenes": [{}],
      "shots": [{}],
      "blocks": [{}],
      "speakers": [{}],
      "textualContentModeration": {
        "id": 0
      }
    }
  }
]

```

Figure 7. Video object in the response JSON from video indexer

2.3.3 Scene segmentation, keyframe, and optical character recognition (OCR)

The application extensively uses Microsoft Video Indexer API for the scene segmentation process. Video Indexer segments a video into temporal units based on structural and semantic properties [12]. The API provides a detailed JSON video insight that contains timestamps for shots, scenes, keyframes, and OCR.

- **Scene:** Video indexer segments a video into scenes based on color coherence across consecutive shots and retrieves the start and end time of each scene. A scene is composed of a series of consecutive shots, which are semantically related [12]. The application uses scene objects from video insight for scene segmentation.

```

"scenes": [
  {
    "id": 1,
    "instances": [
      {
        "adjustedStart": "0:00:00",
        "adjustedEnd": "0:00:03.833",
        "start": "0:00:00",
        "end": "0:00:03.833"
      }
    ]
  },
  {
    "id": 2,
    "instances": [
      {
        "adjustedStart": "0:00:03.833",
        "adjustedEnd": "0:00:10.133",
        "start": "0:00:03.833",
        "end": "0:00:10.133"
      }
    ]
  }
]

```

Figure 8. Scene object in JSON response

- **Shot:** A shot is a series of frames taken from the same motion-picture camera. A shot change is determined based on the visual cues of the video. Shot's metadata contains start time, end time, and list of keyframes within a shot [12].

```

"shots": [
  {
    "id": 1,
    "tags": [
      "Outdoor"
    ],
    "keyFrames": [
      {
        "id": 1,
        "instances": [
          {
            "thumbnailId": "5016843e-d0c2-41c3-b72c-7b79abf774d9",
            "adjustedStart": "0:00:00.167",
            "adjustedEnd": "0:00:00.2",
            "start": "0:00:00.167",
            "end": "0:00:00.2"
          }
        ]
      }
    ],
    "instances": [
      {
        "adjustedStart": "0:00:00",
        "adjustedEnd": "0:00:03.833",
        "start": "0:00:00",
        "end": "0:00:03.833"
      }
    ]
  }
]

```

Figure 9. Shot and keyframe object in JSON response from video indexer

- **Keyframe:** Video Indexer detects the best stable keyframe which represents a shot. Keyframes are the representative frames selected from the entire video based on aesthetic properties [12]. The keyframes are used by the application to generate pythia captions.
- **OCR:** It contains instances of time ranges that show when the text on screen appears in the video [12]. The object contains information like confidence score, height, and width of the OCR rectangle and location of the rectangle.

```

"ocr": [
  {
    "id": 1,
    "text": "Look niöfuFfheV!",
    "confidence": 0.656,
    "left": 230,
    "top": 164,
    "width": 180,
    "height": 19,
    "language": "en-US",
    "instances": [
      {
        "adjustedStart": "0:00:07",
        "adjustedEnd": "0:00:07.5",
        "start": "0:00:07",
        "end": "0:00:07.5"
      }
    ]
  },
  {
    "id": 2,
    "text": "lasses & workout'?"-,
    "confidence": 0.7735,
    "left": 360,
    "top": 164,
    "width": 236,
    "height": 20,
    "language": "en-US",
    "instances": [
      {
        "adjustedStart": "0:00:07",
        "adjustedEnd": "0:00:07.5",
        "start": "0:00:07",
        "end": "0:00:07.5"
      }
    ]
  }
]

```

Figure 10. Ocr object in JSON response from video indexer

2.3.4 Caption generation

Video Indexer extracts the keyframes for each scene of a video. These keyframes metadata contains a thumbnail image URL for each keyframe. These keyframes URLs are publically accessible. The keyframes extracted for a video are sent to the Azure Computer Vision API by invoking REST API. Azure's Computer Vision service analyzes images to detect and provide insights about their visual features and characteristics. The visual features include image type,

description of the image, objects, description, tags, color, faces, adult content, etc. The API generates a description of an image based on the objects identified in the image. The descriptions are then further evaluated and assigned a confidence score for each image. A list of descriptions is then returned, ordered by confidence score, non-ascending [13]. The description for each keyframe with the confidence score is saved in the database. These keyframes descriptions are aggregated to form the description of a scene.

2.3.5 ListenByCode

ListenByCode [18] is a speech to text conversion tool. It uses deep learning neural network algorithms for speech recognition. The video is converted into .mp3 format and uploaded to the ListenByCode API. The API processes the audio and returns a list of timestamps with sentences. This audio data is saved in the database.

2.3.6 Text to speech conversion

The descriptions generated for each scene of a video are converted from text to speech using Google Cloud Speech to text API [15]. This API returns the audio files in .mp3 format. The audio files for a video are then saved in the project's directory in a public/audio folder. These audio files are played along with the original soundtrack of the video.

2.3.7 Environment detection

IBM Watson Max Classifier is used for detecting the environment of the video. The keyframes images generated by Video Indexer are sent to IBM Watson Max Classifier. The model is based on the Places365-CNN Model. This model recognizes the 365 different classes of scene/location in the Places365-Standard subset of the Places2 Dataset. The environment metadata contains information not only regarding whether it is indoor or outdoor but also various categories like street, beach, school, ballroom, forest, auditorium, etc [14]. The current version of the application does not make use of the environment detection.

2.3.8 Database

The application uses Amazon Relational Database Service (RDS) [19] for storage. A PostgreSQL database instance on RDS is created and connected with the application. RDS reduces the time and effort to set up, operate a database in the cloud. It also provides great scalability. It also reduces administrative tasks such as database setup, patching, and backups. RDS provides high availability and security. The video insight extracted by Video Indexer is saved in tables - video, scene, keyframe, shots, blocks. The video description created or edited by the user is saved in the description table.

2.4 Technical Stack

- Cloud service used for project deployment: Amazon AWS EC2
 - Operating System: Ubuntu Server 18.04 LTS 64-bit (x86)
 - Type: t2.micro
 - Number of virtual CPUs: 1 vCPU @ 2.5GHz (Intel Xeon Family)
 - Memory (RAM): 1GB
 - Public IP address: 18.221.192.73
 - Public DNS: ec2-18-221-192-73.us-east-2.compute.amazonaws.com
 - EC2 console instance tag: sfsu_youdescribe
 - Availability Zone: us-east-2c
 - VM Instance Storage: 30GB
 - Mounted EBS instance Storage: 50GB
- Runtime Environment
 - NodeJS : v10.16.3
- Web Development Framework
 - ExpressJs: v4.16.0
- Frontend Technologies
 - Bootstrap
 - HTML 5, CSS
 - Javascript (ECMAScript 6 Standard)

- YouTube Iframe API
- Database: PostgreSQL (Amazon RDS)
 - Type: db.t2.micro
 - PostgreSQL Version: v11.1-R1
 - Allocated Storage: 20GB
 - Storage Type: General Purpose (SSD)
 - Database Port: 5432
- AI Components
 - Microsoft Azure Cognitive Services [28]:
 - Video Indexer API (Video Intelligence API v1.5.1) [12]
 - Computer Vision API v2.0 [13]
 - Text-To-Speech API v1.3.0 [15]
 - IBM Max Scene Classifier v1.1.0 [14]
 - ListenByCode [18]

2.5 Contribution to the Project and Challenges

All the work described in the implementation and user study development section is my contribution to the project.

- Section 3.1: Automation Of Scene Segmentation: developed using publicly available APIs.
- Section 3.2: Merge Scene based on Audio Conflicts: developed using a service created by another team member.
- Section 3.3: OCR Cleanup: cleaned up the OCR by tuning the parameters like location, confidence score.
- Section 3.4: Integration Of Pythia Captioning Service: created an API to generate pythia captions for all the scenes of the video. I used a pythia captioning service created by the team.

- Section 3.5: Filter Description using BLEU Score developed functionality of filtering all the scenes of a video using BLEU score
- Section 3.6: Database Design: designed table structures like video_audio, us_user, us_helper_video, question, answer, scene_qna_mapping, etc.
- Section 4: Development of User Study system: developed the system for conducting user study with the team.
- Conducted User experiments with sighted volunteers.

To design the automation framework for scene segmentation was challenging as it involved multiple steps starting from downloading the video to uploading to Video Indexer, fetching video insight, and saving in the database. For user study 2, the development of an interactive interface was difficult in terms of designing the application which can handle different settings for the user study. Implementing the feature of synching the video on the admin page with respect to the connected user was challenging. For removing similar sentences from the descriptions, I spent a lot of time and effort to understand how to use the BLEU score and what threshold to choose to detect a sentence similarity with reference to other sentences. For choosing the parameters on which the OCR should be filtered, I first spent time in understanding the metadata of OCR received from Video Indexer. I also thoroughly analyzed OCR data for different videos to understand how repetition can be reduced. All this helped in improving my data analysis skills. Conducting user experiments was a good learning experience. It was interesting to get feedback directly from the users. It helped me to think about different features from the user's perspective while implementing it. While working on different parts of the project, I have gained the knowledge of system design, database design, implementation of backend APIs, integration of third-party APIs with the application. Functionalities like merge scenes helped in strengthening my algorithms.

3. IMPLEMENTATION

3.1 Automation of Scene segmentation

The application uses Microsoft's Video Indexer for the scene segmentation. Therefore, it is important to have the integration of Video Indexer with the application.

3.1.2 Sequence Diagram

The sequence diagram shows the interaction between different objects of the application while uploading the video to the application, arranged in a timing sequence.

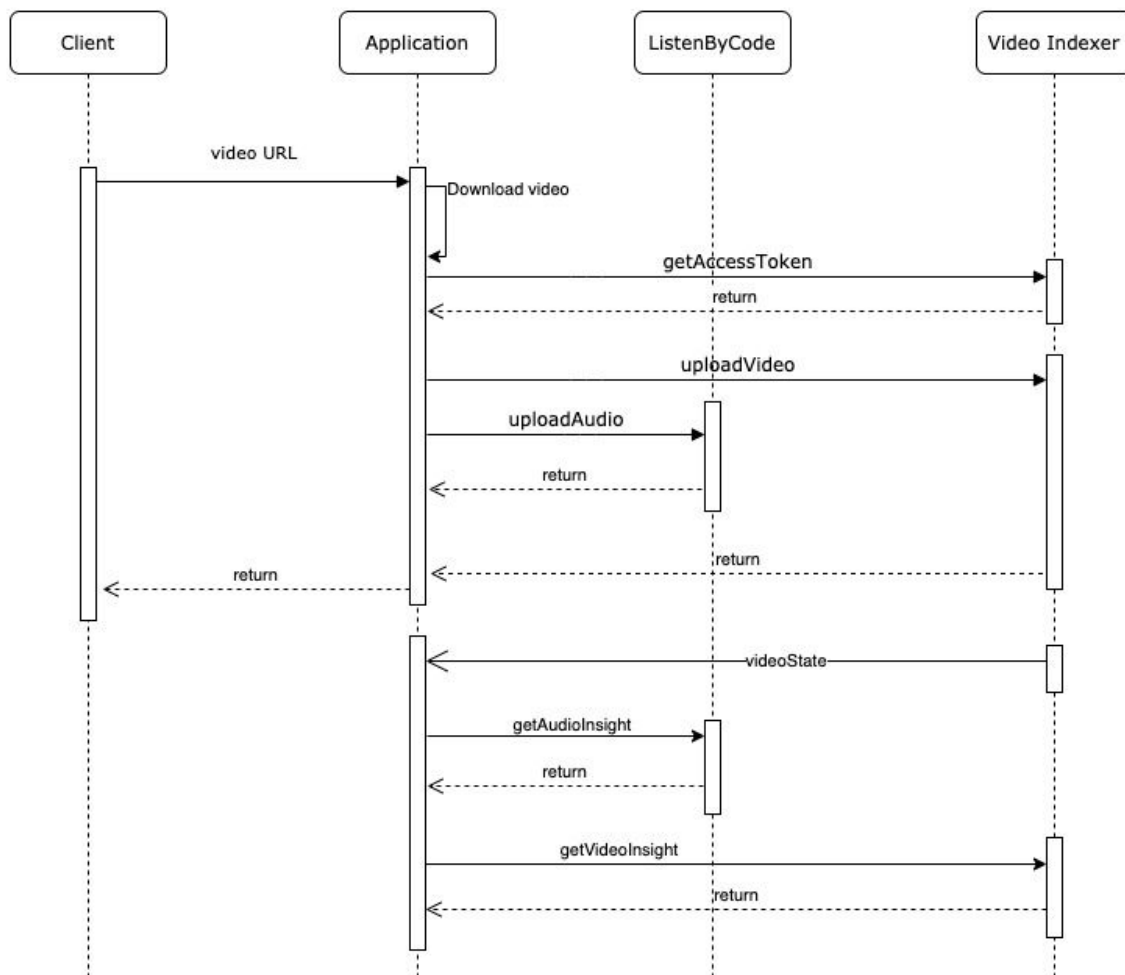


Figure 11. Sequence diagram of uploading a video to the application

3.1.3 Upload Video feature Flow Diagram

Figure 12 shows the flow of the Upload video feature when the user makes a request to upload the video using a youtube URL. The sequence of events is discussed in detail below.

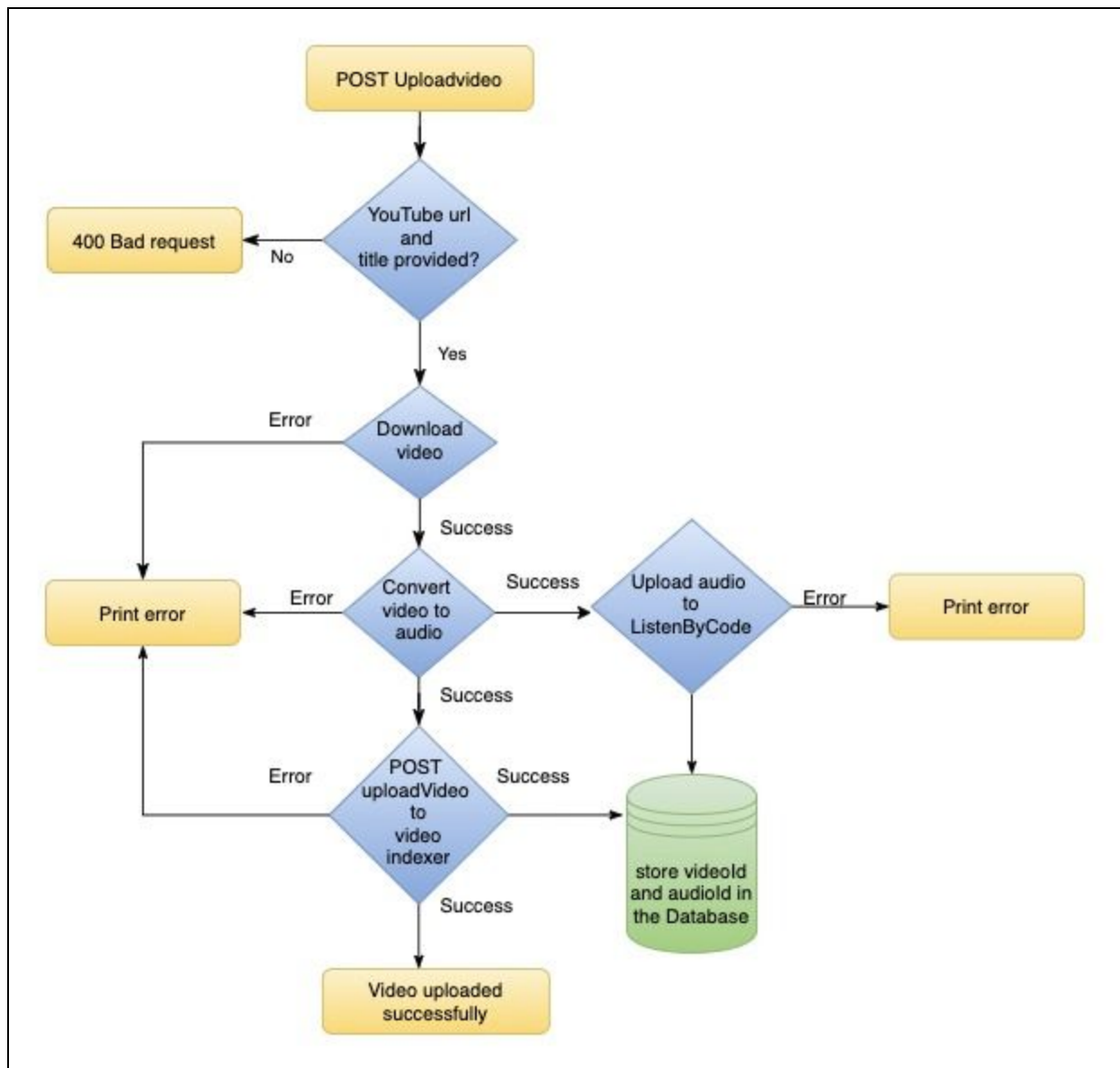


Figure 12. Upload video feature flow diagram

1. The user can upload the video on the Microsoft Video Indexer. The user will provide the youtube URL and title of the video. If any of these two parameters are missing, the user will get a response with a status code of 400 and a message “One or two of the parameters missing”.
2. The video will be downloaded from YouTube using a library, youtube-dl. This is available as a Node.js module. The video will be downloaded in .mp4 format. The video will be saved in the “video” folder of the project’s root directory.
3. The audio will be extracted from the downloaded video. The video will be converted from .mp4 format to .mp3 format.
4. The video will be uploaded to Video Indexer. After a successful upload, the Video Indexer will return an id. This id will be stored in the table “video_audio”. This id will be used to fetch video Insight from Video Indexer after the video is successfully indexed.
5. The audio will be uploaded to the ListenByCode server. On successful upload, ListenByCode server will return an orderid. This order id will be saved in the database table “video_audio”.

3.1.4 Insert Video Data into Database Flow Diagram

Figure 13 shows the flow diagram of inserting a video into database features. The sequence of events that occur when Video Indexer notifies the application after it has successfully processed the video is discussed below.

1. A callback URL is provided while making a post request to upload the video to the Video Indexer. After successful indexing of the video, the Video Indexer hits the endpoint of our application. It supplies an id and state as the query string parameter.
2. The id will be used to track the video in the database. The data of a video is saved with a unique youtube id. This YouTube id is further to play video in the youtube player.

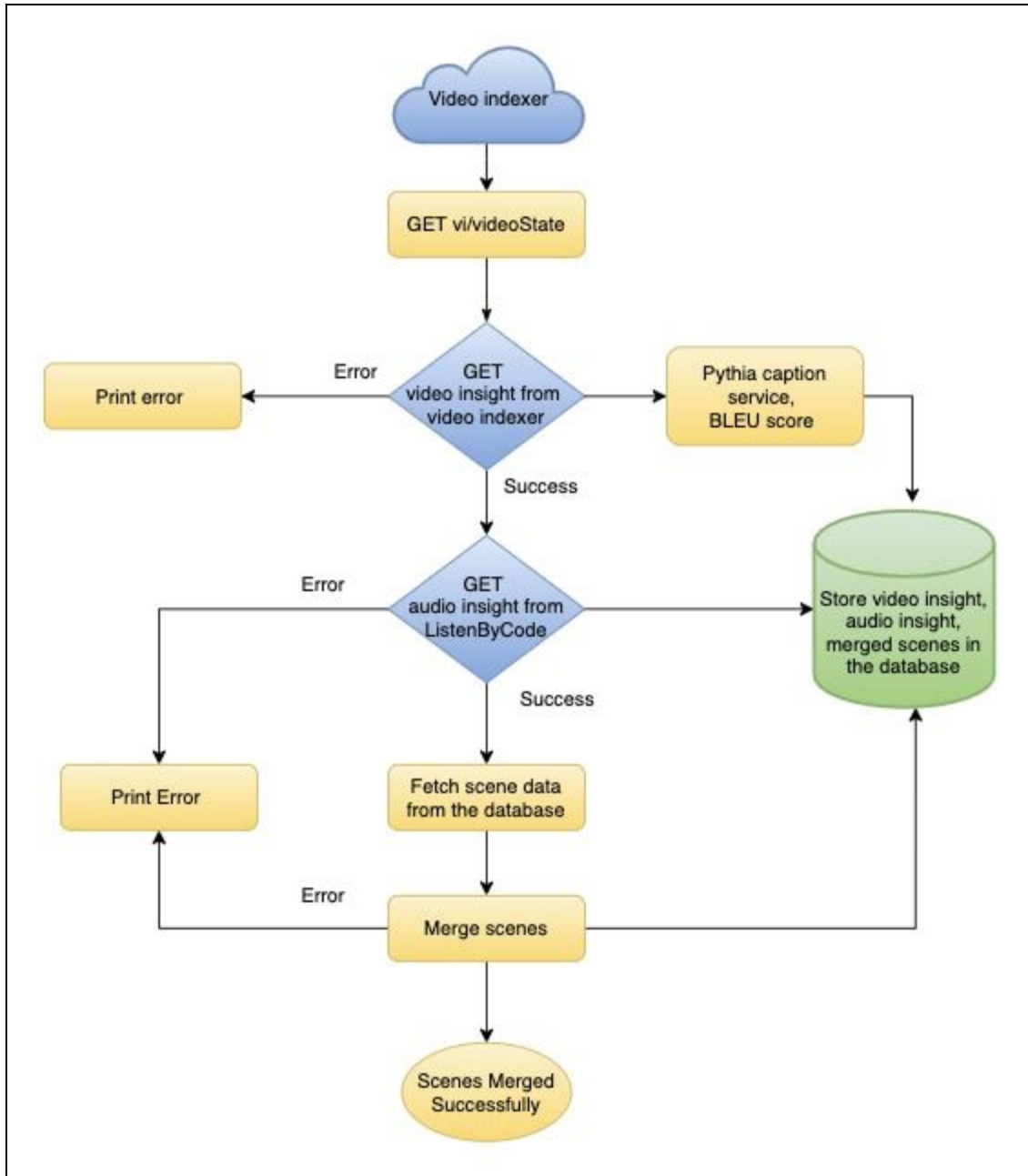


Figure 13: Callback URL hit by video indexer

3. A request is made to the Video Indexer to fetch the video Insight. On the success, the video Insight is parsed and saved in the database in various tables: keyframe, shot, block, scene.

4. The audio Id saved for the video in the “video_audio” table is fetched. This audioId is used to fetch audio data from ListenByCode. The audio data is saved in the video_dialogs table.
5. The scene data is fetched from the “scene” table. The scene data and audio data is used to merge scenes if there are any audio conflicts. The merged scenes are saved in the merge_scene table.

3.1.5 Back End API

- **Upload Video to Video Indexer**

Request URL: **GET** <http://localhost:5001/vi/uploadVideo>

Request parameters

Name	Type	Description
video	string	Required. Video is the url of the youtube video.
title	string	Required. Name of the video

Table 1. Request parameters for upload video API to video indexer

Response

- 200 OK, Message: Video uploaded successfully
- 400 Bad Request, Message: Missing request parameter

- **Microsoft Video Indexer API**

A. Get Account Access Token: fetch account access token which is used for uploading a video to the Video Indexer.

Request URL:

GET

[https://api.videoindexer.ai/Auth/{location}/Accounts/{accountId}/AccessToken\[?allowEdit\]](https://api.videoindexer.ai/Auth/{location}/Accounts/{accountId}/AccessToken[?allowEdit])

Request parameters

Name	Type	Description
location	string	Required. location indicates the Azure region
accountId	string	Required. Account Id of the video indexer.
allowEdit	boolean	Determines the token write permission.

Table 2. Request parameters for get account access token API

Response

- 200 OK, Returns the access token
- 401 Unauthorized, The credentials in the request body are not valid

B. Upload Video

Uploads the video and on its success returns a video Id. Video Indexer starts indexing the video after a successful upload.

Request URL

POST <https://api.videoindexer.ai/{location}/Accounts/{accountId}/Videos?name={name}&privacy={privacy}&callbackUrl={callbackUrl}&accessToken={accessToken}>

Request Parameters

Name	Type	Description
location	string	Required. location indicates the Azure region
accountId	string	Required. Account Id of the video indexer account
name	string	Required. Title or name of the video

Privacy	string	Required. Video privacy mode. Should be public to make thumbnails accessible
callbackUrl	string	Required. Format- URI. On successful completion of videoIndexing, a notification will be sent. id and state will be added to the query string parameters.
Language	string	Optional. Language of the video will be used to generate the transcript.
accessToken	string	Required. Provided in the URL query string parameter or in Authorization header as a Bearer token. The token expires within 1 hour.

Table 3. Request parameters for upload video API to application

Response

- 200 OK, Video uploaded successfully

A globally unique identifier is returned which is used to fetch the video insight from the Video Indexer. This requestid is used by Video Indexer to maintain the logs.

- 404 Not found
- 400 Bad Request, Invalid input
- 401 Unauthorized
- 500 Internal server error
- 504 Gateway Timeout

C. Get Video Index

To get keyframes using the Video Indexer API, upload and index your video using the Upload video call. Once the indexing job is complete, call Get Video Index. This will give you all of the insights that Video Indexer extracted from your content in a JSON file.

Request URL:

GET [https://api.videoindexer.ai/{location}/Accounts/{accountId}/Videos/{videoId}/Index\[?language\]](https://api.videoindexer.ai/{location}/Accounts/{accountId}/Videos/{videoId}/Index[?language])

Request Parameters

Name	Type	Description
accountId	string	Required. Account Id of the video indexer account
videoId	string	Required. video id
language	string	Required. language in which video insight will be translated

Table 4. Request parameters for upload video API to application

Response

- 200 OK, Video index retrieved successfully. Figure 15 below shows the JSON received containing video insight.
- Error: 400 status code.
- If there is an issue while indexing the video. The state of the video will be 'failed'. The video_audio table will have the video_state updated as 'Failed'. Figure 14 shows the output when video indexing fails

```
{
  "partition": null,
  "description": null,
  "privacyMode": "Public",
  "state": "Failed",
  "accountId": "5a8c6ddf-900f-48b4-8a3a-0cd5cf48cb4b",
  "id": "d6b62c9dba",
  "name": "Bawdy Storytelling presents Blythe Baldwin: WINNING!",
  "userName": "Umang Mathur",
  "created": "2020-04-24T21:56:02.9446169+00:00",
  "isOwned": false,
  "isEditable": false,
  "isBase": true,
  "durationInSeconds": 0,
  "summarizedInsights": {},
  "videos": [],
  "videosRanges": []
}
```

Figure 14. Output when video indexing fails

```

{
  "accountId": "SampleAccountId",
  "id": "476892a745",
  "name": "test2",
  "description": null,
  "userName": "SampleUserName",
  "created": "2018-04-25T17:40:28.2732821+00:00",
  "privacyMode": "Private",
  "state": "Processed",
  "isOwned": true,
  "isEditable": false,
  "isBase": true,
  "durationInSeconds": 13,
  "summarizedInsights": {},
  "videos": [
    {
      "accountId": "SampleAccountId",
      "id": "476892a745",
      "state": "Processed",
      "privacyMode": "Private",
      "processingProgress": "100%",
      "failureCode": "General",
      "failureMessage": "",
      "externalId": null,
      "externalUrl": null,
      "metadata": null,
      "insights": {
        "version": "0.9.0.0",
        "sourceLanguage": "en-US",
        "language": "en-US",
        "transcript": {},
        "sentiments": {},
        "blocks": [
          {
            "id": 0,
            "instances": [
              {
                "adjustedStart": "00:00:00",
                "adjustedEnd": "00:00:13",
                "start": "00:00:00",
                "end": "00:00:13"
              }
            ]
          }
        ]
      }
    }
  ]
}

```

Figure 15. Video insight

- **youtube-dl**

youtube-dl is a command-line program to download videos from YouTube or other websites [21]. The npm has a module which uses youtube-dl [22] to download the public unrestricted videos from youtube. It takes the YouTube URL of the video and file format as the input. The video is downloaded in the format of mp4. The video is saved in the “video” folder of the project’s root directory.

- **FFMPEG**

FFMPEG is a framework for processing video and audio. node-ffmpeg is a module for node. This library provides a set of functions and utilities to abstract commands-line usage of ffmpeg. Node-ffmpeg is used to convert an mp4 file to an audio mp3 file [23].

- **ListenByCode API**

ListenByCode is a speech to text conversion tool. It uses deep learning neural network algorithms for speech recognition.

A. Upload audio file to ListenByCode

Request URL:

POST <https://www.listenbycode.com/api/v1/upload>

It uploads the audio file to the Listen by code server. On successful upload, an order id is returned, which is used to fetch audio Insight.

Request body:

Multipart/form-data

```
const formData = {
  appkey: 'yA0zHAY9LQBJQGtshcIGJX368IbC4Enx',
  language: "en",
  file: readStream,
  callbackurl: '',
};
```

Response:

200 OK,

```
{"code":0,"msg":"","status":"success","result":{"orderId":"OIVLXP20200212110759"}}
```

B. Fetch Audio Insight from ListenByCode

Request URL:

GET <http://18.221.192.73:8081/v1/audioclips/getspeechtotextresult>

Request parameters:

Name	Type	Description
orderid	string	Required. orderid is used to fetch audio Insight

Table 5. Request parameters for fetch audio insight from ListenByCode

Response: 220 OK

```

{
  "code": 0,
  "msg": "",
  "status": "success",
  "result": {
    "text": "",
    "sentence": [
      {
        "sentence": " Yeah.",
        "confidence": "0.3400",
        "startTime": 3.64,
        "endTime": 4.76
      },
      {
        "sentence": " excuse me.",
        "confidence": "0.6700",
        "startTime": 6.7,
        "endTime": 7.66
      },
      {
        "sentence": " Do you mind if I a",
        "confidence": "1.0000",
        "startTime": 8.44,
        "endTime": 10.22
      },
      {
        "sentence": " yeah,",
        "confidence": "0.4100",
        "startTime": 10.65,
        "endTime": 11.54
      }
    ],
    "word": {}
  }
}

```

Figure 16. Audio timestamps

3.2 Merge Scene based on Audio Conflicts

The scene segmentation from Video Indexer generates a lot of scenes for some of the videos. We observed that sometimes a scene is chopped in the middle of the dialogue. When the audio description is played for this scenario, it creates issues. The merging of scenes based on audio conflicts will help in handling the edge cases mentioned below for playing the audio description of the scenes.

1. Both start time and end time are in the previous scene
2. Start time in the previous scene and end time in the present scene
3. Start time in the previous scene and end time in the next scene
4. Start time in the present scene and end time in the present scene
5. Start time in the present scene and end time in the next scene
6. Both start time and end times are in the next scene

Implementation

The merge function requires scene data, audio timestamps. When a video is uploaded in the database, then this merge function will be invoked. The function iterates over the audio timestamps and looks for all the timestamps whose start time is between the start time and end time of the current scene. If for any of these timestamps, the end time is greater than the end time of the current scene, then we merge the next scene with the current scene. After merging all the scenes based on audio conflicts, the data is inserted in the merge_scene table. The merge_scene table is the replica of the scene table. When the merge feature merges the scenes, the scenes start, end time, description, ocr is updated. Hence, it is good to have a different table which has updated scene data. There is a column “merged” which contains a list of scene ids. This information helps us to keep a record of the merged scenes. Figure 17 shows the implementation of the merge feature.

In the code snippet of merge function below, the following is the details about the parameters:

- VideoInsight: The scene data for a video is fetched from the database.
- AudioInsight: The audio data from ListenByCode video contains the timestamps of the dialogues.

```
static merge(videoInsight, audioInsight){
  let videoIndex = 0;
  let mergedList = [];
  mergedList.push(videoInsight[videoIndex++]);
  let i = 0;
  while(i < audioInsight.length && videoIndex < videoInsight.length){
    let size = mergedList.length;
    let s1 = mergedList[size-1].start_time;
    let e1 = mergedList[size-1].end_time;
    let description = mergedList[size-1].original_description;
    let ocr = mergedList[size-1].original_ocr;

    let s2 = audioInsight[i].start;
    let e2 = audioInsight[i].end;

    if((s2 >= s1 && s2 <= e1) && (e2 >= s1 && e2 <= e1)) {
      i++;
      continue;
    }

    if(Math.min(s2, e2) >= e1) {
      mergedList.push(videoInsight[videoIndex++]);
      continue;
    }

    //merge condition
    if(s2 >= s1 && e2 > e1) {
      let sceneObj = mergedList[mergedList.length-1];
      let next = videoInsight[videoIndex++];
      let min = Math.min(s1, next.start_time);
      let max = Math.max(e1, next.end_time);
      sceneObj.start_time = min;
      sceneObj.end_time = max;
      sceneObj.original_description = description + next.original_description;
      sceneObj.original_ocr = ocr + next.original_ocr;

      // let arr = sceneObj.merged;
      // arr.push(next.scene_id);
      sceneObj.merged.push(next.scene_id);
      mergedList.splice(mergedList.length-1);
      mergedList.push(sceneObj);
    } else if(s2 >= e1) {
```

Figure 17. Code snippet of merge function

3.3 OCR Cleanup

The video insight from Video Indexer contains an OCR object which contains instances of time ranges that show when the text on screen appears in the video [12]. Figure 18 shows the ocr object. The object contains information like confidence score, height, and width of the OCR rectangle, top is the Y coordinate, and left is the X coordinate of the rectangle. The data had to be filtered as it had a lot of repetitions. For instance, if a text stays on the screen for a longer duration then it is repeated as Video Indexer captures all the time instances of the text on the screen. Sometimes the text on screen is not captured properly like if the font color is too light, there is fading away of the text or text staying for very little time on the screen. All these issues make the cleaning up of the OCR highly essential. In order to reduce the repetition, the system compared two texts. The texts are considered similar if they have the same content and same location, XY coordinates. If the time duration between similar texts is less than 3 seconds then they are ignored. This is already implemented by one of the team members.

```
"ocr": [
  {
    "id": 1,
    "text": "LA",
    "confidence": 0.3467,
    "left": 188,
    "top": 38,
    "width": 21,
    "height": 16,
    "language": "en-US",
    "instances": [
      {
        "adjustedStart": "0:00:02.336",
        "adjustedEnd": "0:00:09.343",
        "start": "0:00:02.336",
        "end": "0:00:09.343"
      }
    ]
  },
  {
    "id": 2,
    "text": "BAWDY",
    "confidence": 0.4439,
    "left": 275,
    "top": 66,
    "width": 110,
    "height": 25,
    "language": "en-US",
    "instances": [
      {
        "adjustedStart": "0:00:00.467",
        "adjustedEnd": "0:00:09.343",
        "start": "0:00:00.467",
        "end": "0:00:09.343"
      }
    ]
  }
]
```

Figure 18. OCR object in the video insight JSON

Despite cleaning up the text, the text still contained a lot of repetition, misspelled words, gibberish. The text is further cleaned up by using information about confidence, location, and duration from the ocr object. In order to further reduce the repetition and improper capturing of the text, the instances for which a text stays on the screen for less than 1 second is ignored. On increasing the time limit, there was a loss of information in many cases. The text is further filtered on the basis of the confidence score. The data was thoroughly analyzed to decide a threshold. The best results were found with a confidence score of 0.56. For one of the videos, ‘Hope for Paws’, there was a lot of repetition of the title of the video in almost every ocr instance. This was handled by setting the top limit for the ocr rectangle. Figure 19, 20 and 21 shows the results after cleaning ocr for three videos.

Video	Scene_num	ORIGINAL OCR	OCR AFTER CLEANING
Hope For Paws	1	Animal Rescue Organization for Please HOPE Subscribe SB	Animal Rescue Organization Subscribe SB
Hope For Paws	2	,Äiä stray dog who wandered into a-yard-and-then-collapsed. I-lepe\$or+aws was notified Of Hope.EofVâPaws was nofified of a yard-anVúthGTCVðllapsed. _ __ -ÄiHope For Paws was notified o a yard-anu,ÄcthvffCvðllV\$PSV©d.	—a stray dog who wandered into Hope.EofÉPaws was nofified of
Hope For Paws	3	Pa and liresponded to the call and ru hed to the location. I responded For Paws For paws O Hope For Paws O Hope The dog was responsive, but clearly sin distress. but, -learly -in distrV©ss. but clearlysiriWdistreys. butuev•ly in distie s. O Hope For but clearly in diktre,Äcs aws O Hope r Paws Hope For Hope & llo showed his teeth to wV\$rn me that he was in a lot of pain. O Hope For aws	and liresponded to the call and ru hed to the location. The dog was responsive, but, -learly -in distréss. llo showed his teeth to wärn me that he was in a lot of pain.
Hope For Paws	4	We created a makeshif t was cooperative. gurney and Hope guirney and was co perative. O Hope For Paws We created a makeshift as cooperati Hope For Paws wa O Hope For He knew we were there toelp im O Hope He knew we were there to help im. For Paws O Hope For Paws For Paws aws Hope For Paws	We created a makeshif t was cooperative. gurney and guirney and We created a makeshift He knew we were there to help im.
Hope For Paws	5	Hope For Paws Paws c Hope For P s We headed to the hospital to figure out what was wrong. O Hope For O Hope	We headed to the hospital to figure out what was wrong.
Hope For Paws	6	O Hope For Paws O Hope For Paws O Hope For Hope For Paws O Hope For Paws X-ray revealed nothing was broken, but he had a serious infection.	X-ray revealed nothing was broken, but he had a serious infection.

Figure 19. Results after cleaning up OCR for video hope for paws.

Video	Scene_num	ORIGINAL OCR	OCR AFTER CLEANING
Bawdy	1	BAWDY STORYTELLING WINNING! True Blythe Baldwin LA	Blythe Baldwin
Bawdy	2	STOR LUNG LUNG LUNG LUNG BAWDY STORYTELLING BAWDY LUNG STO STO ELUNG STORf3ELUNG STO ELUNG STO ELUNG STOR?FELUNG STOR'ÄELUNG STO STOR\$TELUNG LUNG STO STORfiELUNG LUNG STO STOR'	
Bawdy	3	BAWDY LUNG STO STOR7ELUNG STO #ELUNG STO #ELUNG STOR3rELUNG STOR#ELUNG STOR3TELUNG STORfiELUNG LUNG	
Bawdy	4	BAWDY STORYTELLING True For More Information About Bawdy Storytelling Visit: http://www.bawdystorytelling.com LA	For More Information About Bawdy Storytelling Visit:

Figure 20. Results after cleaning up OCR for video bawdy storytelling

Video	Scene_num	ORIGINAL OCR	OCR AFTER CLEANING
Fried Potato Ball	1	put potatoes in watuv¶¶nrd,Äcuoil them put potatoes in them put potatoes in watuv¶¶nrd,Äcuoil them put potatoes in them after abouvo minutes, they are cooked (Checkitawi!tln a- knife)'. after abouvo 3q minutes, then tney are cooked (Cheek it withea (Check itv•with av¶¶k-n.ilfv©)Z. @heckit with a (Check<it wiltltn a (checklt a knife)... ool drain them anVd-let cool	put potatoes in them after abouvo 3q minutes, then tney are cooked (Cheek it withea drain them anVd-let cool
Fried Potato Ball	2	cool then put them into a bowl peeled and chopped	then put them into a bowl peeled and chopped
Fried Potato Ball	3	Fv•Yd?d softened butter... oo softened hutter... (0.8112 'text: ' 'FâYd?d softened butter...' ' diff:' 1.44) ...a pinch of salt... oo 00 ...and black pepper. the entire world!	FâYd?d softened butter... oo ...a pinch of salt... 00 ...and black pepper.
Fried Potato Ball	4	,ÄÇThe best spice of the entire world! the entire world! the entire world!	•The best spice of the entire world! the entire world!
Fried Potato Ball	5	crush and mix with a fork Now cover and let cool	crush and mix with a fork Now cover and let cool
		at least 2 hours in the refrigerator Now cover and let cool After this time, take a teaspoon of dough After this time, take a teaspoon of.d6ugh After this time, take a teasppu offv•v•u(gliz- give shape ball shape give ball shape and cover with flour Bv©*aJen egg, with a,Äçvüinch of salt; r ,ÄiBv©aten egg, with a,Äûpinch of*alt ...a pinch of salt... Andlbrehdcrumbss And breadyv°mbs;q Finish shv•ping and lev•ikv© Onjyplate Finish ,sha+ing and plate, Finish shv•pihg and- lqa.v.v©\.,Äçvðnl@plate Finish shapmg a plate Finish shaping	at least 2 hours in the refrigerator Now cover and let cool After this time, take a teaspoon of dough After this time, take a teaspoon of.d6ugh give ball shape and cover with flour And

Figure 21. Results after cleaning up OCR for video fried potato ball

3.4 Integration Of Pythia Captioning Service

Pythia Captioning service generates a caption for an image. The service is developed by one of the team members. The service internally uses Pythia. Pythia is an open-source, deep learning framework for vision and language multimodal research [24]. It is built on top of the Pytorch machine learning framework. It generates captions for the image and answers the questions related to the image based on the visual data.

The system currently uses Azure Computer Vision API to generate captions for the scenes of a video. We want to use Pythia captions for creating descriptions. The video insight received from the Video Indexer contains the keyframe images which will be sent to the Captioning service. The response from the service will be aggregated to form the scene description.

Implementation

- The system uses a video indexer's vision model to extract insight from the video. The video insight contains :
 - Scene segmentation: video is chopped down into a number of scenes based on visual cues. A scene is composed of a series of consecutive shots. Figure 22 shows scene metadata.
 - Shot detection: A shot is a series of frames taken from the same motion-picture camera. A shot change is determined based on the visual cues of the video. Shot's metadata contains start time, end time, and list of keyframes within a shot [12]. Figure 23 shows shot metadata.
 - Keyframe extraction: VI detects the best stable keyframe which represents a shot [12].

```

"scenes": [
  {
    "id": 1,
    "instances": [
      {
        "adjustedStart": "0:00:00",
        "adjustedEnd": "0:00:09.71",
        "start": "0:00:00",
        "end": "0:00:09.71"
      }
    ]
  },
  {
    "id": 2,
    "instances": [
      {
        "adjustedStart": "0:00:09.71",
        "adjustedEnd": "0:08:46.76",
        "start": "0:00:09.71",
        "end": "0:08:46.76"
      }
    ]
  },
  {
    "id": 3,
    "instances": [
      {
        "adjustedStart": "0:08:46.76",
        "adjustedEnd": "0:14:18.991",
        "start": "0:08:46.76",
        "end": "0:14:18.991"
      }
    ]
  }
]

```

Figure 22. Scene metadata from video indexer

```

"shots": [
  {
    "id": 1,
    "keyFrames": [
      {
        "id": 1,
        "instances": [
          {
            "thumbnailId": "dd93ce53-a037-40e8-a83d-4454fbf7b26a",
            "adjustedStart": "0:00:00.567",
            "adjustedEnd": "0:00:00.6",
            "start": "0:00:00.567",
            "end": "0:00:00.6"
          }
        ]
      }
    ]
  },
  {
    "id": 2,
    "instances": [
      {
        "thumbnailId": "51696700-163f-4225-b266-03778a3c38b9",
        "adjustedStart": "0:00:00.868",
        "adjustedEnd": "0:00:00.901",
        "start": "0:00:00.868",
        "end": "0:00:00.901"
      }
    ]
  }
]

```

Figure 23. Shot metadata containing keyframes

- The video insight is available as the JSON object. The insight is parsed and inserted into various tables in the database. The 'KeyFrame' table contains the metadata of each keyframe. The column 'thumbnail_url' contains the URL of the image.
- To generate Pythia captions for a video, firstly a list of scenes is fetched from the scene table and then keyframes images for those scenes from the keyframe table. These

keyframes images are then sent to the Pythia Caption generation service. The response is updated for each keyframe in the column 'pythia_caption' of the keyframe table. These keyframes are then aggregated to form the description of a scene and updated in the 'pythia_caption' column of the Scene table. Figure 24 and 25 shows the implementation of the pythia caption generation process.

```
static async fetchPythiaCaption(image_url){
  try{
    let result = await axios({
      method: 'post',
      url : 'http://ec2-34-221-19-208.us-west-2.compute.amazonaws.com:8080/api',
      data: qs.stringify({
        token : 'ZSDL6MXc6urYauWzVLXnUgE8yZcfCncCYgNyX4jRRnrGCTLmV4KUKrNDYSbfWAf2',
        image_url
      }),
      headers: {'Content-Type': 'application/x-www-form-urlencoded'}
    })
    return result;
  }catch(error){
    console.log(error);
    throw error;
  }
}
```

Figure 24. Code snippet for post request to pythia captioning service

```

class PythiaCaptionGenerationController {
  static async generatePythiaCaption(videoId){
    try{
      let data = await keyframeModel.getKeyFramesByVideoId(videoId);
      let captionArr = [];
      for(let i=0; i< data.length; i++){
        let thumbnail_url = data[i].thumbnail_url;
        let keyframe_id = data[i].keyframe_id;
        let captionResponse = await this.fetchPythiaCaption(thumbnail_url);
        data[i].updatedPythiaCaption = captionResponse.data;
        captionArr.push({keyframe_id, thumbnail_url, caption: captionResponse.data });
      }

      let response = await keyframeModel.updatePythiaCaption(captionArr);
      let pythiaCaptions = await keyframeModel.getKeyFramesByVideoId(videoId);

      let sceneMap = new Map();
      for(let i=0; i< data.length; i++){
        let sceneId = data[i].scene_id;
        let caption = data[i].updatedPythiaCaption;

        let value = sceneMap.get(sceneId);
        if(value){
          if(!value.includes(caption)){
            caption = value + '.' + caption;
            sceneMap.set(sceneId, caption);
          }
        }else{
          sceneMap.set(sceneId, caption);
        }
      }

      let sceneCaptionArr = [];
      for (let [key, value] of sceneMap) {
        console.log(key + ' = ' + value)
        sceneCaptionArr.push({sceneId:key, caption:value});
      }
      await sceneModel.updatePythiaCaptionForAllScenes(sceneCaptionArr);
      console.log('Successfully updated pythia caption in scene table');
    }catch(e){
      console.log(e);
    }
  }
}

```

Figure 25. Code snippet of generating pythia caption for a video

API to generate captions for a video

Request URL

GET <http://localhost:5001/pythia/generateCaption>

Request parameters

Name	Type	Description
videoId	string	Required. videoId of the video

Table 6. Request parameter for generate pythia caption API

Response

- 200 OK
Message: Pythia captions successfully updated
- 400 Bad Request
Message: Missing request parameter
- Error message with 4XX status code

3.5 Filter Description using BLEU Score

The application uses Microsoft Azure Computer Vision API to generate descriptions of the scene. The description of the scene contains a lot of similar sentences. We need to filter out these similar sentences as they do not contribute much in describing the scene. In order to filter out, we use the BLEU algorithm.

BLEU or Bilingual Evaluation Understudy [20] is an algorithm for evaluating the quality of text translated by a machine with that of a human. The algorithm compares a candidate translation with one or more reference translations and provides a score. The score is between 0 and 1. The score indicates the similarity between the candidate text and the reference text. The more the value is closer to 1 represents a high similarity between texts.

Implementation

An application is created using Express Node JS. The application is hosted on AWS. It runs on port 3005. The application spawns a process to run a python script which calculates the BLEU score between a candidate text and one or more reference text. FilterBaselineDescriptionController class contains a method filter that takes video id as input. Firstly, the description of all the scenes is fetched from the 'scene' table for the video. The description consists of one or many sentences separated by a new line. The paragraph is split into sentences by using '\n' as the delimiter. All these sentences are then added to a Queue. A sentence is polled from the queue until the queue is not empty. The sentence polled is sent as a candidate and the rest of the sentences in the queue are sent as the reference sentences to the BLEU score API. The BLEU score API returns the score of the candidate sentences with respect

to the reference sentences between 0 and 1. The score is the average of One gram, Two grams, Three grams, and Four grams scores. If the score of the sentence is less than or equal to 0.56, it is pushed to an array. An array is created for every scene which stores all the approved sentences. All these sentences are then aggregated and the 'description_blue' column is updated with this new description. Figure 26 and 27 show the code of FilterBaselineDescriptionController class.

```
100, 2 minutes ago | 1 author (100%)
class FilterBaselineDescriptionController{

  static async filter(videoId){
    try{
      let sceneData = await sceneModel.getSceneData(videoId);
      for(let i=0; i < sceneData.length; i++){
        let scene = sceneData[i];
        let desc = await this.calculate(scene.original_description);
        scene.description_bleu = desc;
        console.log({scene});
      }

      // update Bleu description in scene table
      let res = await sceneModel.updateBleuDescription_Scene(sceneData);
      console.log({res});
      return res;
    }catch(e){
      console.log(e);
      throw e;
    }
  }
}
```

Figure 26. Code snippet of filter function of FilterBaselineDescriptionController class

```

static async calculate(description){
    let result = [];
    let rejected = [];
    let queue = description.split('\n');
    if(queue.length == 1){
        result.push(queue[0]);
        return result;
    }

    while(queue.length > 1){
        let sentence = queue.shift();
        let body = {
            sentence,
            reference: queue
        }
        try {
            let res = await axios.post('http://localhost:3005/score', body);
            let score = res.data;
            if(score < 0.56){
                result.push({sentence, score});
            }else{
                rejected.push({sentence , score});
            }
        }catch(e){
            console.log(e);
        }
    }
    result.push({sentence: queue[0]});
    let desc = '';
    result.forEach(element=>{
        desc = desc + element.sentence + '\n';
    });

    return desc;
}

```

Figure 27. Code snippet of calculate function of FilterBaselineDescriptionController class

Results

The output after running the BLEU score on the description of three videos:- one minute time machine, Fickey Hall, and Hope for Paws is shown in Figures 28, 29 and 30. The best results were obtained when the BLEU Score threshold is set to 0.56. Anything above 0.56 gives almost similar meaning sentences.

The comparisons of sentences shown in table seven helped in setting up the threshold for the BLEU score. As the score increases the sentences become more and more similar in the meaning. Hence after going through a lot of data, a score of 0.56 showed good results.

Candidate Sentence	Reference Sentence	BLEU Score
a person sitting on a park bench	a person sitting on a bench	0.747
a bedroom with a bed and desk in a small room	a bedroom with a bed and window in a room	0.66
a close up of an engine	a close up of a cat	0.598
a man and a woman looking at the camera	a man looking at the camera	0.514
a person sitting at a bus stop	a person sitting at night	0.499
a woman sitting on a bench in a park	a man sitting on a bench	0.411
a person wearing a hat	a person sitting at night	0.179
a close up of a person	a man looking at the camera	0.0416

Table 7. BLEU score of candidate sentence with respect to reference sentence

- The BLEU score sometimes gave inaccurate results for some of the sentences.

Candidate Sentence	Reference Sentence	BLEUScore
Brian Dietzen et al. looking at the camera	Brian Dietzen looking at the camera	0.20
a close up of cat	a close of clock	0.801

Table 8. Inaccurate BLEU score of candidate sentence with respect to reference sentence

- The name of the person is the same but because of the presence of 'et al'. The score is very low.
- A score of 0.80 is calculated for totally different sentences- the cat and clock have no similarity.

Original description	Bleu description
a person lying on a bed a person lying on a bed a group of people in a room a small white dog looking at the camera a blurry image of a dog a dog looking at the camera a brown and white dog looking at the camera	a person lying on a bed a group of people in a room a blurry image of a dog a dog looking at the camera a brown and white dog looking at the camera
a dog lying on a bed a close up of a dog a person sitting next to a dog a woman sitting on a bed a person in a green room a dog looking at the camera a person holding a dog a person sitting on the floor a blurry image of a person a close up of a hand a person in a white shirt a dog lying on a bed a person taking a selfie a person holding a dog a dog sitting on a bed a dog looking at the camera	 a person sitting next to a dog a person in a green room a person sitting on the floor a blurry image of a person a close up of a hand a person in a white shirt a dog lying on a bed a person taking a selfie a person holding a dog a dog sitting on a bed a dog looking at the camera

Figure 29. Description after running BLEU score on the video hope for paws

Original description	Bleu description
Fickes Hall Residence	
a large room a view of a room a dining room table a room filled with furniture and a fireplace a living room filled with furniture a room filled with furniture and a large window	a large room a view of a room a dining room table a living room filled with furniture a room filled with furniture and a large window
a building with a fence in front of a window a bench in a park a bedroom with a bed and desk in a small room a bedroom with a bed and window in a room a living room a bedroom with a bed and desk in a small room a view of a city street a car parked on the side of a building	a building with a fence in front of a window a bench in a park a living room a bedroom with a bed and desk in a small room a view of a city street a car parked on the side of a building
a car parked on the side of a building a house that has a sign on the side of a road a dark city street	a car parked on the side of a building a house that has a sign on the side of a road a dark city street

Figure 30. Description after running BLEU score on video Fickes hall residence

APIs

- **Filter Description using BLEU score**

Request URL

GET <http://localhost:5001/filterDescriptionBleu>

Request parameters

Name	Type	Description
videoId	string	Required. videoId of the video

Table 9. Request parameters for filter description API to application

Response

- 200 OK
Message: Data updated for the video in Scene table
- 400 Bad Request
Message: Missing request parameter

- **Calculate the bleu score of a candidate sentence with a reference sentence**

Request URL

POST <http://18.221.192.73:3005/score>

Request parameters

Name	Type	Description
sentence	string	Required. sentence to be compared
reference	array of string	Required. a list of sentences which will be used as reference

Table 10. Request parameters for calculate BLEU score API

Response:

200 Ok, Message: BLEU Score between 0 and 1. Figure 31 shows the response from the API.

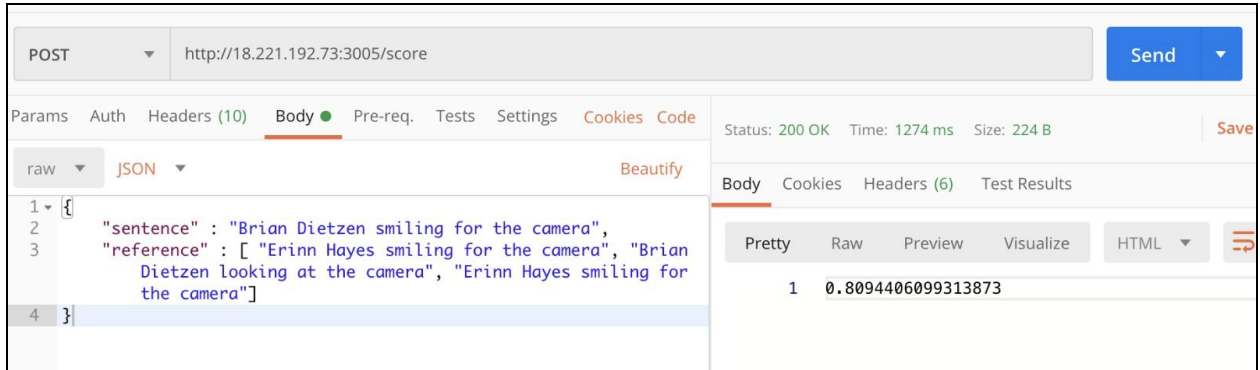


Figure 31. Screenshot of POST request from postman

3.6 Database Schema

The database schema of the application is shown in figure 32.

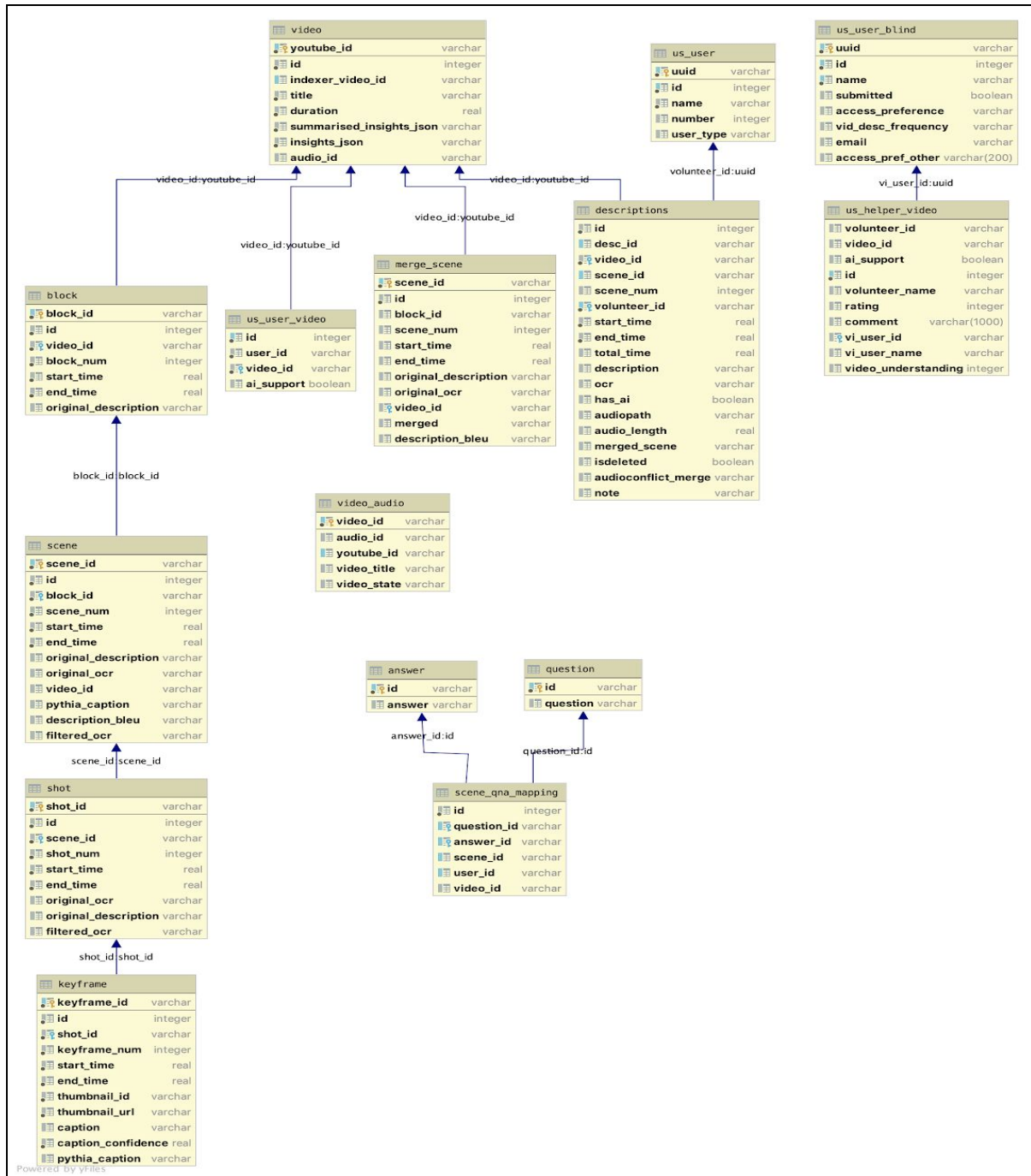


Figure 32. Database schema

The database consists of the tables mentioned below :

- **Video:** The table consists of the videos processed by the Microsoft Video Indexer. The video is tracked by the youtubeId. This youtube id is used by the Youtube player. The indexer_video_id can be used to track the video logs in the video indexer.

▼  video

 id	integer = nextval('video_n_i...')
 youtube_id	varchar
 indexer_video_id	varchar
 title	varchar
 duration	real
 summarised_insights_json	varchar
 insights_json	varchar

- **Video_Audio:** The table consists of the lists of the videos which are sent to the Video Indexer for processing. The column 'video_id' is a unique id returned by the Video Indexer. This is used to fetch video insights. The 'audio_id' is the orderId returned by the ListenByCode API. This id is used to fetch audio timestamps from ListenByCode.

▼  video_audio

 video_id	varchar
 audio_id	varchar
 youtube_id	varchar
 video_title	varchar
 video_state	varchar

- **Scene :** A video is chopped down into scenes. Each scene has a start time and end time. The original_description contains the caption generated by Microsoft Azure Vision API. The pythia_caption contains the caption generated by the Pythia model.

▼	scene
id	integer = nextval('scene_n_i...')
scene_id	varchar
block_id	varchar
scene_num	integer
start_time	real
end_time	real
original_description	varchar
original_ocr	varchar
video_id	varchar
pythia_caption	varchar
description_bleu	varchar

- **Description:** The scene data modified by the user is saved in this table. The table contains the file path of the audio files generated for each scene. The merge_scene column saves the list of sceneIds merged in a scene.

▼	descriptions
id	integer = nextval('descripti...')
desc_id	varchar
video_id	varchar
scene_id	varchar
scene_num	integer
volunteer_id	varchar
start_time	real
end_time	real
total_time	real
description	varchar
ocr	varchar
has_ai	boolean
audiopath	varchar
audio_length	real
merged_scene	varchar
isdeleted	boolean
audioconflict_merge	varchar

- **Scene Question Answer Mapping:** The question answers created for a scene are mapped using this table. QuestionId and answerId is the foreign key to the question and answer table. Separate question and answer table provides the flexibility of saving one or more answers for one question.

scene_qna_mapping	
id	integer = nextval('scene_upd...')
question_id	varchar
answer_id	varchar
scene_id	varchar
user_id	varchar
video_id	varchar

- **Video_dialogues:** The table consists of the audio timestamps and sentences generated by the ListenByCode API for a video.

video_dialogs	
start_time	double precision
end_time	double precision
id	integer = nextval('video_dia...')
video_id	varchar
sentence	varchar
audio_id	varchar

- **KeyFrame:** Shots consist of keyframes. The keyframes for a shot are saved in this table. A keyframe can be tracked to a shot using 'shot_id'. The thumbnail_url contains publicly accessible url of an image.

keyframe	
id	integer = nextval('keyframe_...')
keyframe_id	varchar
shot_id	varchar
keyframe_num	integer
start_time	real
end_time	real
thumbnail_id	varchar
thumbnail_url	varchar
caption	varchar
caption_confidence	real = 0
pythia_caption	varchar

- **Question:** It consists of a list of all the questions.

question	
id	varchar
question	varchar

- **Answer:** It stores all the answers tables.

answer	
id	varchar
answer	varchar

- **Merge_Scene:** The scenes are merged based on the audio conflicts. The merged scenes data is stored in this table. The structure of this table is same as the scene tale.

merge_scene	
id	integer = nextval('merge_sce...')
scene_id	varchar
block_id	varchar
scene_num	integer
start_time	real
end_time	real
original_description	varchar
original_ocr	varchar
video_id	varchar
merged	varchar
description_bleu	varchar

- **User:** It contains information of all the users in the system.

us_user	
id	integer = nextval('us_user_i...')
uuid	varchar
name	varchar
number	integer
user_type	varchar

- **User_helper_video:** It contains the rating provided for each video by a blind user. The data is related to User study phase 2 with blind and visually impaired users.

us_helper_video	
volunteer_id	varchar
video_id	varchar
ai_support	boolean
id	integer = nextval('test_vide...')
volunteer_name	varchar
rating	integer
comment	varchar(1000)
vi_user_id	varchar
vi_user_name	varchar
video_understanding	integer = 0

- **User_blind:** It contains the details of visually impaired and blind users who participated in User Study phase 2 like name, email Id. The submitted contains the status of ‘user study’ of each user. This was used to send gift cards to the users.

▼  **us_user_blind**

	id	integer = nextval('us_user_b...')
	uuid	varchar
	name	varchar
	submitted	boolean = false
	access_preference	varchar
	vid_desc_frequency	varchar
	email	varchar
	access_pref_other	varchar(200)

- **Shot:** The scene is made up of consecutive shots. The shots metadata for each scene is saved in this table.

▼  **shot**

	id	integer = nextval('shot_n_id...')
	shot_id	varchar
	scene_id	varchar
	shot_num	integer
	start_time	real
	end_time	real
	original_ocr	varchar
	original_description	varchar

4. USER STUDY DESIGN AND DEVELOPMENT

4.1 User Study I - Phase I

Phase 1 of the User Study, evaluates the usage of the system for the describers who had no previous training or experience in describing videos. There were two setups for the experiment:

- 1) AI-supported System: Participants were provided with preprocessed machine-generated scene segmentation, description of the scenes, and text on screen. The participant was allowed to edit the descriptions.
- 2) System with no AI support: Participants had to create scenes manually and describe each scene without any assistance.

4.1.1 User Study Preparation

- The user study was conducted at the University of San Francisco, California. User Study involved human subjects, therefore approval from IRB [25] was required. After getting IRB approval, an advertisement email was sent to the students of San Francisco State University and the University of San Francisco. The students who showed interest were sent a link where they can book a time slot. Once they book the time slot, a confirmation email along with the venue details was sent to the participants.
- To conduct user experiments, cooking videos were selected as these were highly requested by visually impaired and blind users for video description. Two cooking videos were selected for the experiment. It was made sure that videos were similar to keep a fair comparison. Most of the cooking videos generally have music playing in the background and ingredients showing up on the screen. Therefore we picked two videos that had background music with no dialogue. ‘Text on screen’ feature of the User Interface came in handy for the description generation.
- At the application’s database level, users were added, tables were created to save the user’s data for each video.
- A trial run of the experiment was conducted with a volunteer.

4.1.2 Experimental Design

Twenty-two people participated in the User Study. I was part of the team that conducted user experiments.

- Every participant was first asked to go through and sign a consent form which had details about the experiment. After the participant signed the paper, they were assigned a computer system. Then, they were asked to watch a video tutorial on how to use the description software.
- Participants were given two videos to describe, one with AI support and others without AI support User Interface. The AI-supported Interface had scene segmentation, description, and captions. Participants had to edit the descriptions. In the non-AI supported Interface, the participant had to create scenes on their own and describe it.
- After completing the video description, the participants were asked to fill out a questionnaire to understand their experience with the description software. Participants also filled out a NASA-TLX survey - a workload assessment tool [26]. After filling out the forms, every participant was interviewed and the responses were recorded with the Participant's consent.

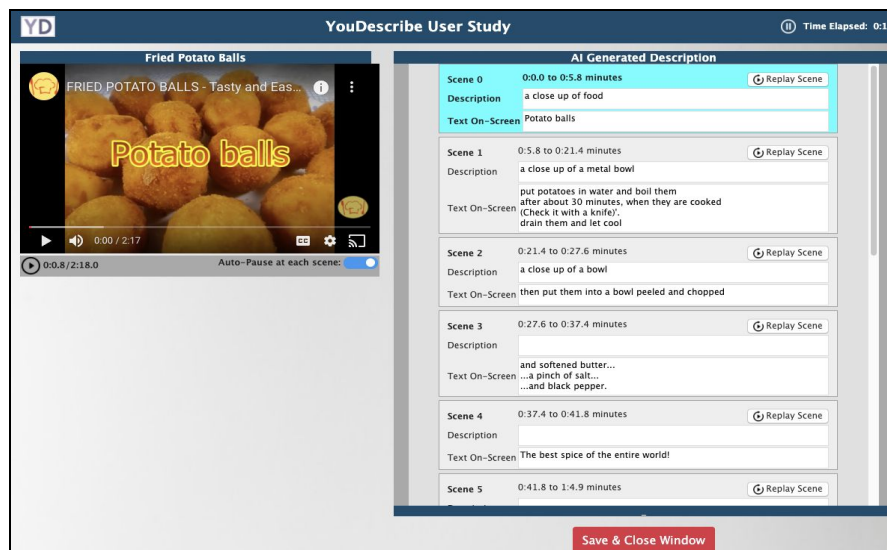


Figure 33. AI-supported interface for sighted volunteers

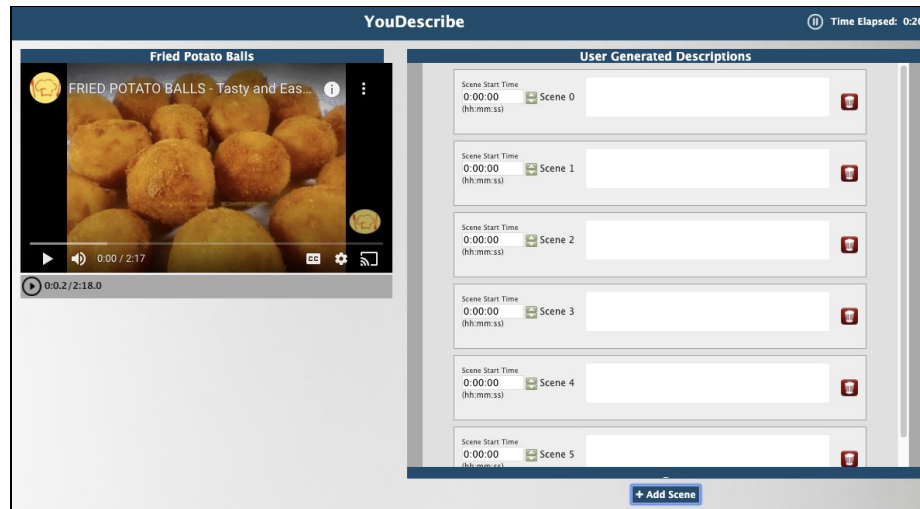


Figure 34: User interface with no AI support for sighted volunteers

Results:

- The Questionnaire data showed that the describers preferred an AI-supported interface and found it faster with Interface with no AI assistance.
- The results of the NASA-TLX survey showed that the participants found the AI-supported interface to be less mentally demanding and requires less effort. The participants found the scene segmentation created by AI-supported Interface very helpful. As per the interview data, Sixteen out of 22 participants found the AI-supported Interface helpful and easier to use because it provided guidelines [2].

4.1.3 User Interface

4.1.3.1 Home Page

- Homepage shows the list of participants for the user study. Developed the Homepage and integrated it with the backend.
- On clicking on any of the participants on the list, the page redirects to the video list page.
- Developed the logic for routing the user to their respective video list page.

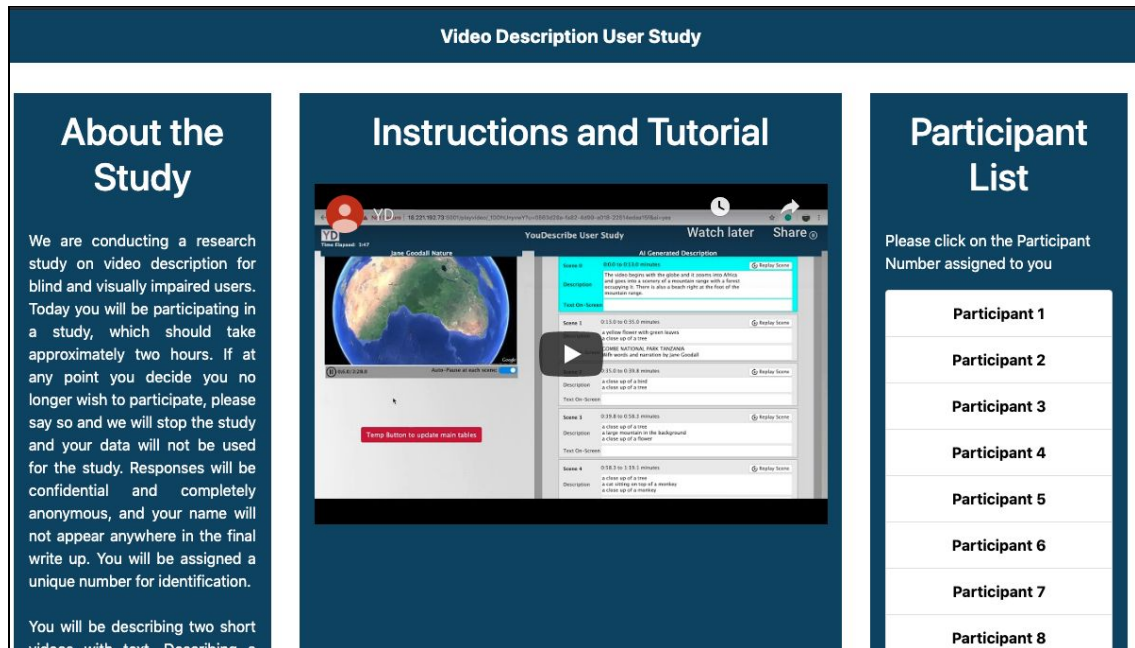


Figure 35. Home page

4.1.3.2 Video List page

- Created a page that consists of two videos selected for each user.
- The videos displayed are in a predefined order, one with AI support and other videos without AI support.
- Developed the frontend page with the video links.

Video Description User Study

Note: Please Ensure That You Attempt the Video Descriptions in Sequential Order Only (i.e, Video 1 Before Video 2)

☐


Video 1: Crispy Cabbage and Carrot Fritters (With Interface Support)

☐


User Survey for Video 1 (Google Form)

☐


NASA TLX Survey for Video 1 (Not Clickable – You Will Be Given a Physical Form to Fill Out)

☐


Video 2: Fried Potato Balls (Without Interface Support)

☐


User Survey for Video 2 (Google Form)

☐


NASA TLX Survey for Video 2 (Not Clickable – You Will Be Given a Physical Form to Fill Out)

☐


General Survey (Google Form)

Figure 36. Video list page for a user

4.1.4 Backend APIs

- Created a user table to manage different users for the system. The columns of user table consist of:
 - Id: unique id to track each record, primary key
 - Uuid: universal unique id given to the user
 - Name: name of the user
 - User_type: tracks the type of the user - describer, visually impaired user, admin.
 - Number: for tracking different phases of a different user study
- Created a table for recording the videos for every user. The columns created:
 - Id: primary key
 - User_id : user id, foreign key from the us_user table
 - Video_id : unique id for a video, foreign key from video table.
 - Ai_support : to identify if the video should be played with AI support or without AI support
- List of APIs developed:
 - addUser: adds a user to the us_user table

- o getAllUsers: fetches all the users from us_user specific to user study phase 1.
- o getUserById: fetches the user from us_user table of a particular user id
- o getVideoByUserId: fetches all the videos of a user from us_user_video table
- o addVideo : inserts the video data in us_user_video

4.2 User Study I - Phase II

In Phase 2 of the User Study, the blind and visually impaired users evaluated the quality of the descriptions generated by the participants in Phase 1 using AI-supported and non-AI supported systems. These descriptions were further converted into speech and played along with the video.

4.2.1 Experimental Design

Twelve visually impaired and blind users participated in the experiment. All participants were provided remote access to the system. From the previous user study, we collected 44 video descriptions from 22 describers (2 videos each). Six participants were given videos produced by the first eleven describers and six participants were given videos by the last 11 describers.

The ratio of the 22 videos given to each participant with AI-supported and with no AI support Interface was 1:1. The videos were split evenly into Video 1 and Video 2. The order of the videos was randomized. Participants did not know if the video is generated with AI support or without an AI-supported Interface. After watching each video, participants were asked to rate the quality of description on a scale of 1 to 5 ('Poor' to 'Excellent'), their understanding of the topic in the video, and any other comments they had about the video [2].

Result

The participants rated the quality of the video higher for the description generated by the AI-supported system as compared to the descriptions generated by the system with no AI support.

4.2.2 Video Rating Page

- The page was developed for blind and visually impaired users which reads the screen using a screen reader. Therefore, special attention was paid in terms of accessibility. Every radio button provided on the page had **ARIA** labels [27].
- The page consists of a list of video links and options for rating each video with a comment section for others.
- The page has the option of entering the email id, which was used to send the gift card to the users participating in the user study.
- The page had two general survey questions regarding video descriptions, which was further used in the user study statistics.
- Developed the logic to route to a page where they can hear the video with the descriptions when a video link is clicked.
- When the user selects one of the options for the questions asked for each video, other options are unselected.

Video Description User Study

Thank you for participating in this user study. Please answer the questions below, and then watch the videos, and answer the questions about each video. There are 22 videos, and each video is about 2 minutes long. The 22 videos are in fact only 2 different videos described by 11 different describers. Even though they are repetitive, please listen to each description, rate the description quality, and leave comments for each video. At the end, there is a button confirming that you completed all the videos. For each video, there is a link to the video on YouTube. Open the link and click the play button. The YouTube video will be played along with the audio description.

Please play each video once all the way through to the end (there are no pause or rewind functions). You are welcome to take a break in between videos. If the Youtube video fails to load, please click the refresh button on your browser. Thank you again for participating.

Please enter your email address:

How do you typically prefer to access images?

☐ With magnification

☐ Rely on description

☐ Tactile graphic

☐ Other :

How regularly do you use video descriptions?

☐ Never

☐ Very infrequently (a few times a year)

☐ Sometimes (a few times a month)

☐ Very Frequently (several times a week)

☐ Almost daily or daily



Video 1: Fried Potato Balls

Please rate the description of this video on a scale of 1 to 5, with 1 being the poorest and 5 being the best:

☐ 5 - Excellent

☐ 4 - Good

☐ 3 - Average

☐ 2 - Fair

☐ 1 - Poor

I feel I have a good understanding of the topic covered in this video due to the video.:

☐ Strongly Agree

☐ Agree

☐ Neutral

☐ Disagree

☐ Strongly Disagree

Comment:



Video 2: Fried Potato Balls

Please rate the description of this video on a scale of 1 to 5, with 1 being the poorest and 5 being the best:

☐ 5 - Excellent

☐ 4 - Good

☐ 3 - Average

☐ 2 - Fair

☐ 1 - Poor

Figure 37. Video rating user interface

4.2.3 Backend APIs

- Created table `us_helper_video` for storing the rating of each video given by a user. The columns created are:
 - `id` : primary key
 - `video_id` : id of the video
 - `volunteer_id` : volunteer id is the user id who described a video
 - `volunteer_name` : name of the volunteer who described the video
 - `rating`: rating for each video
 - `video_understanding` : rating for video understanding
 - `ai_support`: if the description of the video has to be played with AI support or without AI support

- List of API created:
 - `getHelperVideo`: It fetches the video data from `us_helper_video`
 - `updateStatus`: It updates the status of a user in the `us_user_blind` table, once the user submits the rating for all the videos.
 - `updateVIUserInfo`: It updates the user data in the `us_user_blind` table with all the data collected like email id, survey questions.
 - `getUsers_VIStatus`: get the status of the users from `us_user_blind` table
 - `getAllUser_VI`: get count of users from the `us_user_blind`

4.3 User Study II

User study two evaluates the effectiveness of an interactive user interface as compared to a passive user interface for blind and visually impaired users. The user study was unfortunately not conducted. The system developed for the user study will be used in the future.

4.3.1 Experiment setup

The experiment consisted of four different settings, in which the blind and visually impaired users will be asking questions to a chatbot and human. The user will not have any idea if he/she is talking to humans or to a machine. The user will rate the response of each question. The data collected from this study will also be used for training the Visual Dialogue model. The system was developed for Prep session and Setting C.

1. **Prep session:** Prep session is created to make the user acquainted with the system. In this session, the user will ask five or more questions related to a video. These questions will be posted to the admin. The admin will decide if a question is content-based or visual-based. All the questions will be saved in the database.
2. **Session I:** In this session, the user will be asked to watch one or more videos. The video will be paused automatically, and the supplementary description will be played.
3. **Session II:** In the second session, the user will be asked to listen to two or more videos. If the user has any questions while listening to the video, then the user will activate the chatbot by pressing keys: ctrl+Q. The video will be paused, and the user can ask questions. The visual dialogue will answer the question. The user will be asked to rate the response of each question.
4. **Session III:** In this session, the user will watch one or more videos and will ask questions based on visual content. When a question is asked, the video will be paused and the question will be recorded. The question will be converted from speech to text and sent to the admin (human) on the server-side. The admin will type the answer to the question which will be converted from text to speech. The user will be able to hear the answer.

4.3.2 Video Page With AI

To conduct User Experiments, we need to collect questions and answers for the videos.

The describer will be asked to create Q&A for each scene of a video. In order to assist describers, a question-answer generation model is used. These Q&A data will be further used to train the Visual Dialogue model.

Features:

- Create More questions: When the user clicks on the “create more questions” button, then two textboxes are generated for entering questions and answers. The questions and answers are saved in the database for each scene.
- Delete button: When the user clicks on “create more question” or “autogenerate questions”, a delete button is created for each question, answer. When the user clicks on delete, a delete request is made to the server and the questions are deleted.
- Done with the scene: when the user clicks on the button, the video starts playing for the next scene.
- Autogenerate questions: when the user clicks on autogenerate button, the scene description is sent to the Question Generation model. The model creates the questions-answers, which are populated on the user interface in the textboxes. The text boxes are provided so that the user can edit the questions and answers.
- Question generation model: The question generation API was accepting the request in the form data. The request type was changed from form data to JSON data format for easy parsing of data between application servers.
- Merge with the next scene: The user can merge the current scene with the next scene. The merge scene button will combine the description and OCR from the next scene and update it in the description box of the current scene. The end time of the current scene will be updated and the next scene will be deleted from the UI. For each scene, the number of scenes merged is tracked.

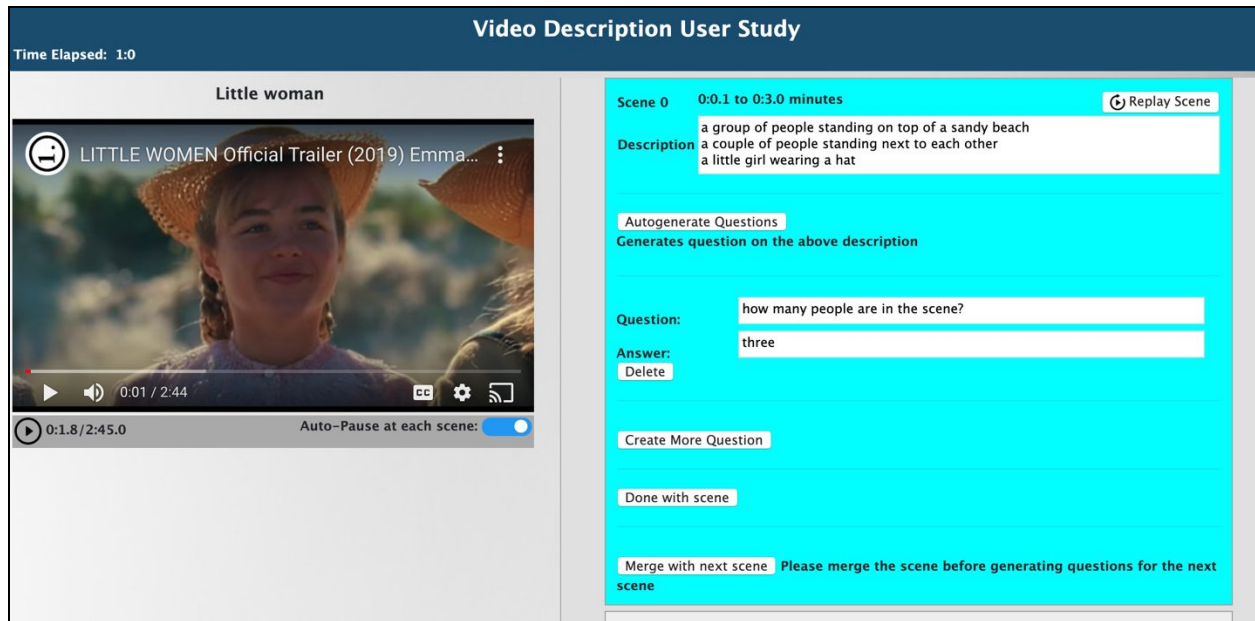


Figure 38. Video page with AI support

Backend APIs

- The previous system did not save the data in the description table. The logic was added to save all the data of a video with scenes with all the modifications to be stored in the database.
- The previous system did not show the data updated by the user. The logic was added to show the modified description for a scene.
- question table: Created question table for storing all the questions:
 - o Id: unique question id
 - o Question: description of the question
- answer table: created answer table for storing all the answers:
 - o id: unique answer id
 - o answer: description of the answer
- scene_qna_mapping table: created a table to keep track of questions and answers generated for a scene for a particular video and by a user.
 - o id : unique id, primary key
 - o question_id: question id from the question table
 - o answer_id: answer id from the answer table

- o scene_id : scene id from the scene table
 - o video_id : video id from the video table
 - o user_id : user id from the user table
- List of APIs created:
 - o **addQuestionAnswer**: It saves the question and answers in the descriptions table.
 - o **getQuestionById**: It fetches all the question from the question table by question id
 - o **getQuestionAnswerByUserId**: fetch question and answers from the descriptions table filtered by user id
 - o **updateQuestion**: updates the question in the question table
 - o **updateAnswer**: it updates the answer in the answer table
 - o **deleteQuestionById**: deletes the question from question table by question id
 - o **deleteSceneById** : delete a scene from scene table by id
 - o **updateSceneEndTime**: update the scene end time in description table after merge
 - o **Fetch scene data for a video**

Request URL: GET <http://18.221.192.73:5001/videoData>

Request parameter

Name	Type	Description
videoid	string	Required. youtube id of the video
userId	string	Required. user id of the describer

Table 11. Request parameters for fetchSceneData API

Response:

The response JSON will consist of :

- OriginalData: this contains original data from table “Scene”
- modifiedData: this contains modified data from table “Description”.
- qnAList: contains the list of questions and answers for a video created by a user.

Each question object contains scene id.

```
{
  "youtube_id": "7QZNpS0uos4",
  "indexer_video_id": "4a1a74b4f5",
  "title": "best scene(The Notebook)",
  "duration": 279.446,
  "originalData": [
    {
      "sc_id": "7QZNpS0uos4_0_1",
      "sc_num": 1,
      "start_time": 0,
      "end_time": 6.131,
      "original_ocr": "",
      "original_description": "a person wearing a hat\nRachel McAdams et al. 1 the camera\na close up of a man and a woman looking at the camera\nRachel McAdams et al. that are looking at the camera",
      "deletedScene": []
    },
    {
      "sc_id": "7QZNpS0uos4_1_4",
      "sc_num": 4,
      "start_time": 40.541,
      "end_time": 81.206,
      "original_ocr": "",
      "original_description": "a man sitting on a stage\na man sitting in a da person wearing a costume\na man looking at the camera\na group of peop standing in front of a crowd\na person standing in front of a crowd\na people looking at each other\na couple of people that are talking to e",
      "deletedScene": []
    }
  ],
  "modifiedData": [
    {
      "questionId": "1a87b181-76b5-11ea-ad10-eb0cc3c0c482",
      "answerId": "1a87b182-76b5-11ea-ad10-eb0cc3c0c482",
      "sceneId": "7QZNpS0uos4_1_4",
      "question": "",
      "answer": ""
    }
  ],
  "qnaList": [
    {
      "questionId": "1a87b181-76b5-11ea-ad10-eb0cc3c0c482",
      "answerId": "1a87b182-76b5-11ea-ad10-eb0cc3c0c482",
      "sceneId": "7QZNpS0uos4_1_4",
      "question": "",
      "answer": ""
    }
  ]
}
```

Figure 39. Response of fetchSceneData

o Save scene data into the Database

The API can be used to save or update the scene data. The API accepts the scene data in the form of an array. The scenes are inserted in the scene table and if sceneId already exists then it is updated. This can be used to update one or more than one scene.

Request URL : POST <http://18.221.192.73:5001/saveAiDescription>

```
Body: {
    User_id,
    videoId,
    Scene_arr: [ ],
    qnaData: [ scene_id: {question} ]
}
```

Response:

On success: status 200, success message

On failure: error will be printed

```

    user_id: "9fe1e2c6-5cd2-11ea-bc55-0242ac130003"
  ▶ scene_arr: (28) [{...}, {...}, {...}, {...}, {...}, {...}, {...}, {...}
    time_taken: 55
  ▼ qnaData:
    ▶ 7QZNpS0uos4_0_1: (2) [{...}, {...}]
    ▶ 7QZNpS0uos4_1_4: [{...}]
    ▶ __proto__: Object
  videoId: "7QZNpS0uos4"
  ▶ __proto__: Object

```

Figure 40. Response of save scene data API

```

▼ scene_arr: Array(28)
  ▼ 0:
    scene_id: "7QZNpS0uos4_0_1"
    modified_description: "a person wearing a hat,Rachel McAdams et al. lookin
    modified_ocr: ""
    start_time: 0
    end_time: 6.131
    has_ai: true
  ▶ deletedScene: []

```

Figure 41. Scene object in the response from save scene API

```

▼ qnaData:
  ▼ 7QZNpS0uos4_0_1: Array(2)
    ▼ 0:
      question: "Who wearing a hat?"
      answer: "person"
      answerId: "d2d85641-721e-11ea-8dc6-13320d006198"
      questionId: "d2d85640-721e-11ea-8dc6-13320d006198"
      ▶ __proto__: Object
    ▼ 1:
      videoId: "7QZNpS0uos4"
      question: "Test Question 1"
      answer: "Test Answer 1"
      ▶ __proto__: Object
      length: 2
      ▶ __proto__: Array(0)
  ▼ 7QZNpS0uos4_1_4: Array(1)
    ▼ 0:
      question: ""
      answer: ""
      answerId: "1a87b182-76b5-11ea-ad10-eb0cc3c0c482"
      questionId: "1a87b181-76b5-11ea-ad10-eb0cc3c0c482"
      ▶ __proto__: Object
      length: 1

```

Figure 42. qnaData in the response from save scene API

qnAData: It is an array containing key/value pairs, where key is scene Id , value is an array of questions for a particular scene. The API will check if the question object does not contain the ids, it will create a new id and then insert it in the table. If the question id and answer id is present, then it will update the question-answer.

- **Merge Scene API:** The API “Save modified scene data into the Database” is also used for merging scenes. To merge the scenes, in the current scene’s deleted array the next scene id is added. This deleted array is used to track the number of scenes merged and will also be used to delete the next scene. Figure 43 shows the merge logic on the front end.

For Ex: Scene 2 is merged in Scene 1. On the front end, start time, end time, description and ocr is changed for Scene 1.

- Scene 1 Start Time: MIN(Scene1, Scene 2) start time
- Scene 1 End time : MAX(Scene 1, scene 2) end time
- Scene 1 OCR: scene 1 + scene 2
- Scene 1 Description: scene 1 + scene 2
- Scene 1 deletedScene: Push scene 2 id in the array.
- The deleted scene array is used to update a merged_scene column in the Scene table which has a list of scene Ids, which are merged in a particular scene. As the scene 1 is merged in scene 2, therefore the scene 2 will be deleted from the description table. Description table contains modified data of each user

```
let newStartTime = Math.min(currSceneData.start_time, nextSceneData.start_time);
let newEndTime = Math.max(currSceneData.finish_time, nextSceneData.finish_time);
currSceneData.finish_time = newEndTime;
currSceneData.start_time = newStartTime;
console.log('New time', newStartTime, '->', newEndTime);

document.getElementById('time:${curSceneId}').innerHTML = `${millisToMinutesAndSeconds(newStartTime)} to ${millisToMinutesAndSeconds(newEndTime)} minutes`
// console.log(allSceneArr);
console.log({nextSiblingSceneId});
let nextSceneDescElement = document.getElementById('s_d_' + nextSiblingSceneId);
let nextSceneOCRElement = document.getElementById('s_o_' + nextSiblingSceneId);
console.log({nextSceneDescElement});

curSceneDescElement.value += '\n' + nextSceneDescElement.value;
curSceneOCRElement.value += '\n' + nextSceneOCRElement.value;
// document.getElementById(curSceneDescElement+'.'+'txt-scene-description').autogrow();
nextSibling.remove();
//delete next scene
let jsonBody = {
  "sceneId": nextSiblingSceneId
};
```

Figure 43. Code snippet of merge logic on the front end

4.3.3 Home page

- The participant and the admin will be able to connect to different sessions using a link.
- These links will connect the participant and the admin to a chatroom where the admin will be able to see all the questions posted by the participant and reply to those questions.

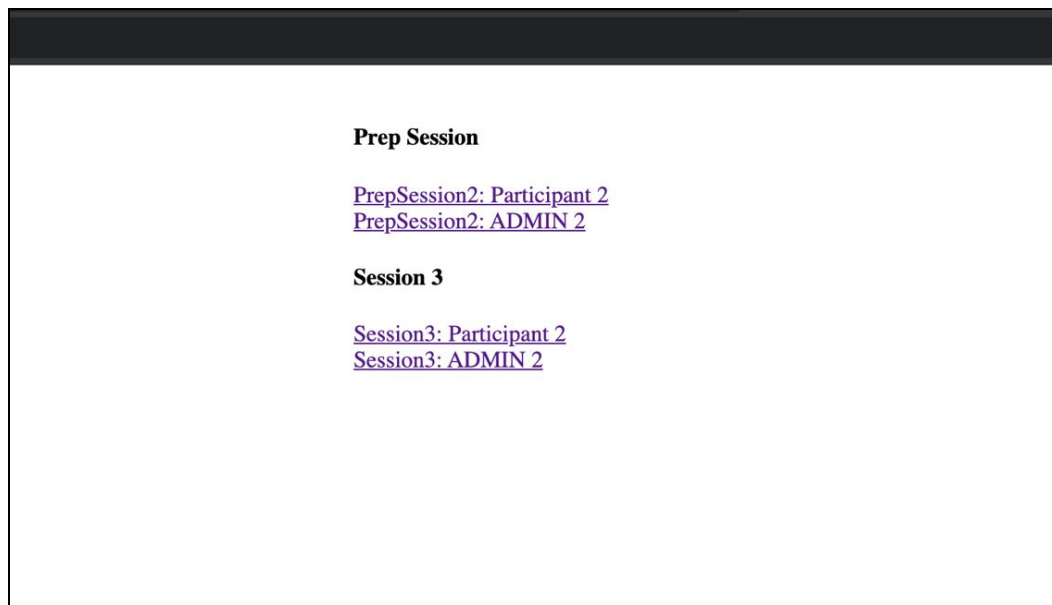


Figure 44. Homepage

4.3.4 Prep Session

The prep session is created to make the user familiar with the system. The user will ask five or more questions related to the video. These questions will be converted from speech to text and posted to the admin. The admin will decide if a question is content-based or visual-based. All the questions will be saved in the database.

Prep Session consists of two pages: one for the participant and another for admin.

4.3.4.1 Prep Session - Participant page

- Ask question button: When the user clicks on “ask question” the user will speak out the question. The question will be recorded until there is a long pause. Once the question is recorded, the question will be displayed in a textbox on the UI. The user can edit the question.
- Submit question button: When the user clicks on the “submit question” button, the question will be saved in the database and also will be visible on the admin page. Once the question is submitted, the button will be disabled in order to avoid submitting the same question again.
- End session button: The user will hear a message and will be prompted to ask at least five questions in case the number of visual-based questions is less than 5.

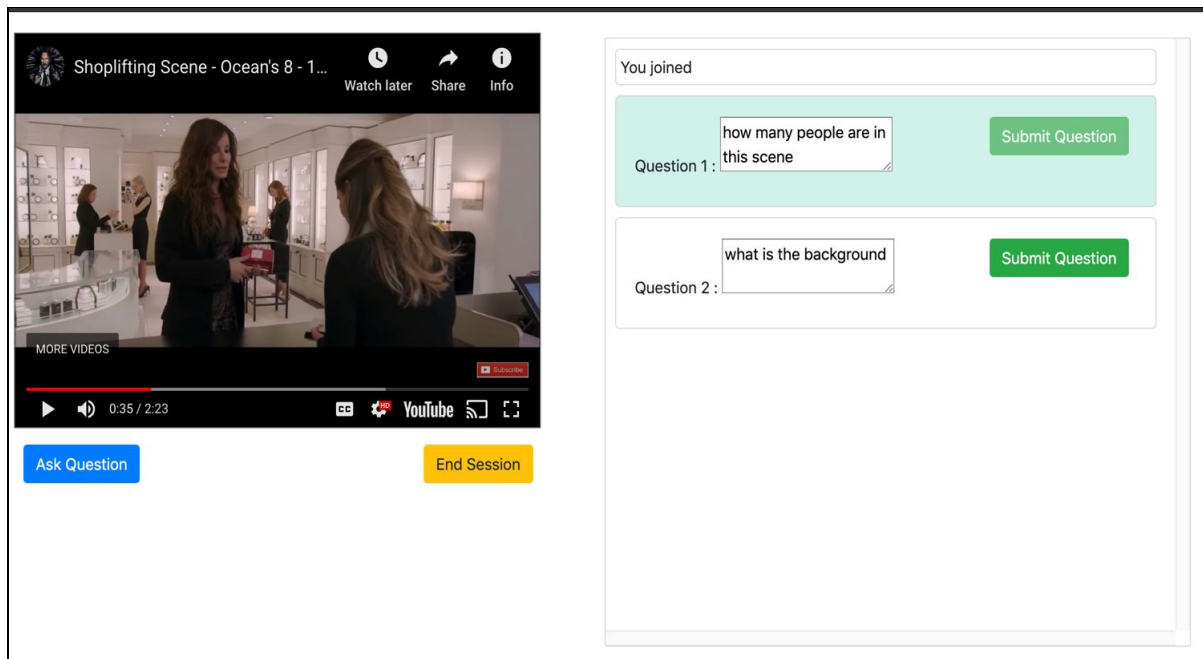


Figure 45. Participant user interface

4.3.4.2 Prep Session – Admin page

- When the participant will pause the video, the admin will receive the message with the participant id and the timestamp.

- The video on the admin page will be in sync with the participant video. If the participant pauses the video, in addition to receiving the message with the timestamp, the admin's video will also stop at the same timestamp.
- When the admin will select the option of “content-based”, the participant will hear a message- “Question not accepted. Please ask a visual-based question”.
- When the admin will select the option of “visual-based”, the participant will hear a message- “question submitted. Please create more questions”.

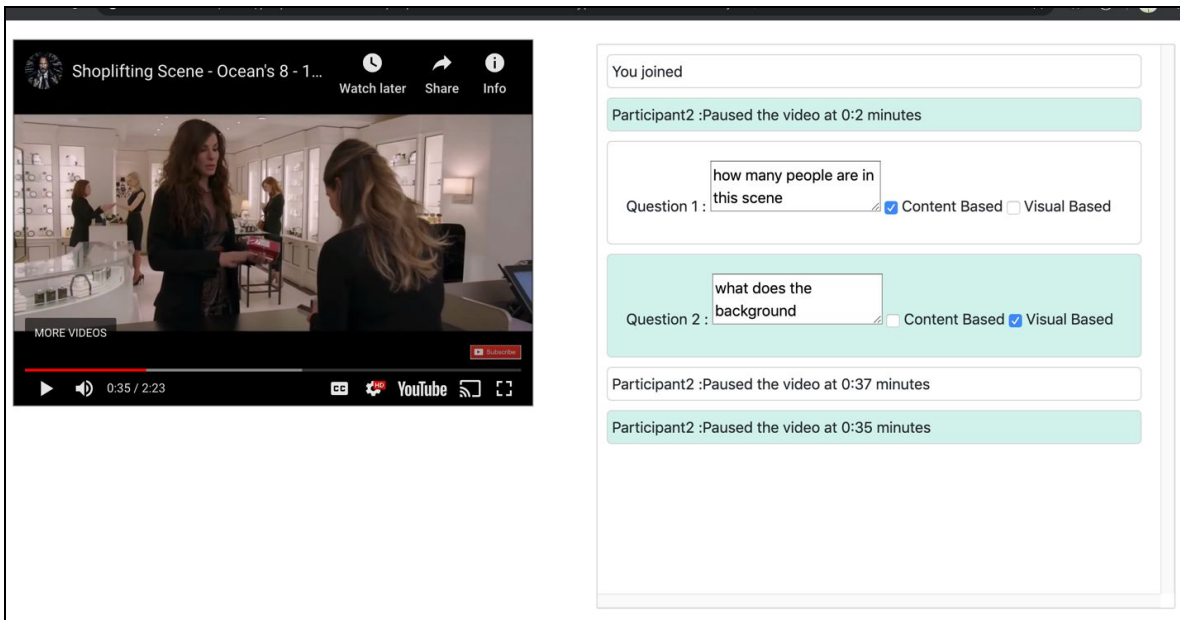


Figure 46. Admin user interface

4.3.5 Session III

4.3.5.1 Participant page – Session III

- The questions asked by the participant in the prep session will be displayed on the UI according to the timestamp. The video will be paused.
- Ask question button: When the user clicks on “ask question” the user will speak out the question. The question will be recorded until there is a long pause. Once the question is recorded, the question will be displayed in a textbox on the UI. The user can edit the question.

- Submit question button: When the user clicks on the “submit question” button, the question will be saved in the database and also will be visible on the admin page. Once the question is submitted, the button will be disabled in order to avoid submitting the same question again.
- The user has to rate the answer by using the rating field.
- End session button: The user will hear a message and will be prompted to ask at least five questions in case the number of visual-based questions are less than five.

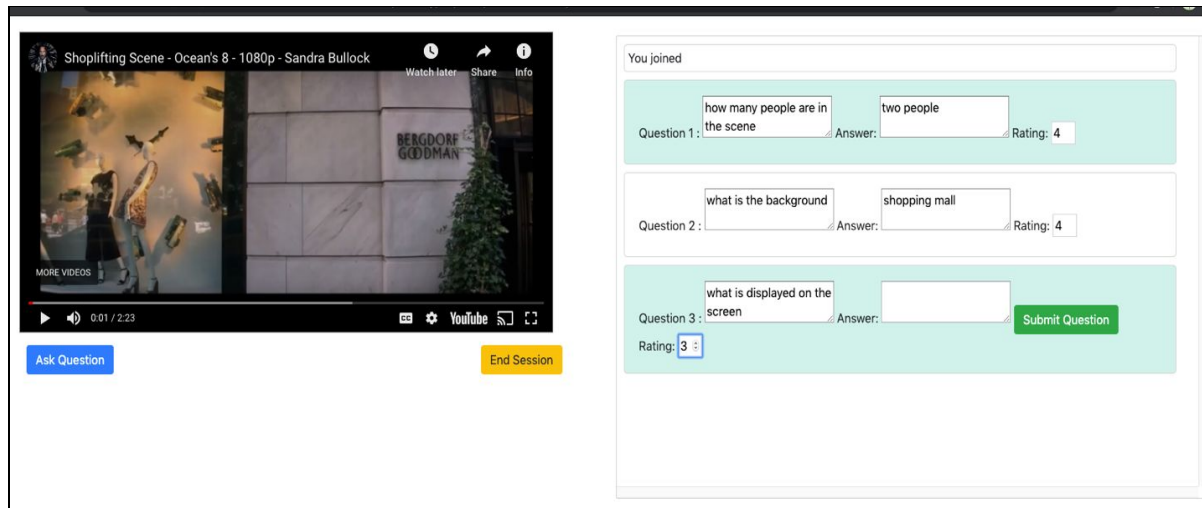


Figure 47. Session III- Participant user interface

4.3.5.2 Admin page - Session III

- The questions asked by the participant are populated on the admin page, the textbox is provided to write the answer.
- Send button: when the send button is clicked, the participant will hear the answer written in the textbox.
- The admin can see the timestamp when the user pauses the video
- The video on the admin page will be in sync with the participant’s video. If the user stops the video, the video on the admin page will be paused at the same timestamp.

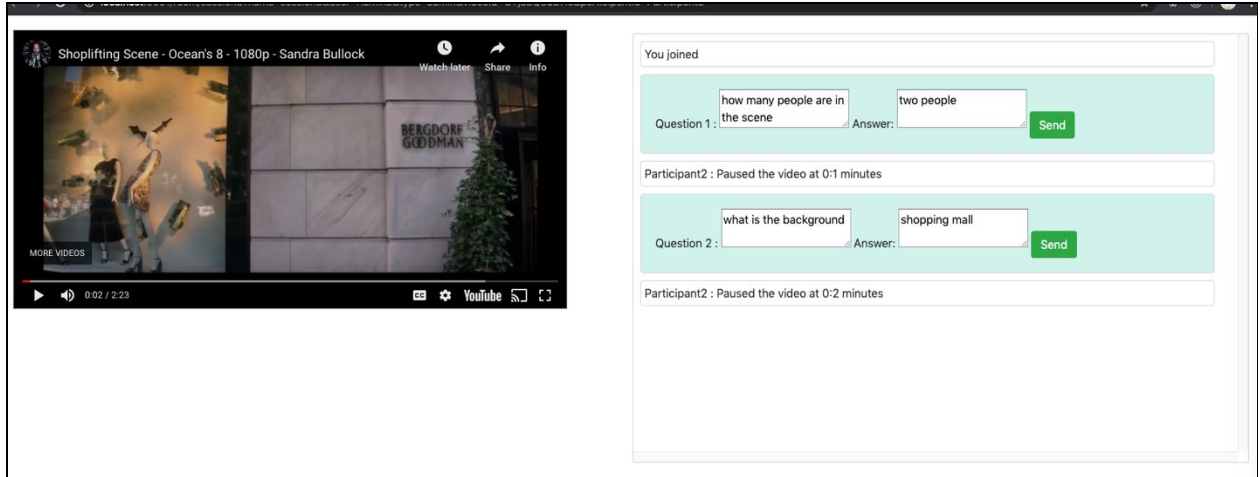


Figure 48. Session III- Admin user interface

4.3.6 Backend APIs

Following are the list of APIs created for the User study II:

- o addQuestionAnswer
- o getQuestionAnswerByUserId
- o deleteQuestionById
- o updateRating
- o updateQuestionType
- us3_session_qna table: created table to map question answer with each session for a user
 - o id: primary key
 - o question_id: question id from the question table
 - o answer_id: answer id from the answer table
 - o user_id: user id from the user table
 - o session_type: type of the session
 - o video_id: id of the video from the video table
 - o time: timestamp of the question
 - o question_type : type of the question- content, visual-based
 - o rating: rating of the answer

5. CONCLUSION

An application is designed and developed to increase video accessibility for visually impaired and blind users. The application helps the sighted volunteers, especially amateur describers, to create high-quality video descriptions with a less cognitive workload. Thereby, improving the quality of description, reducing the turnaround time and hence the cost of the video description. All these factors help in increasing the number of videos with audio description and thereby increasing the video accessibility. The application makes use of various machine learning APIs to automate scene segmentation and description generation. The application provides flexibility to the user in controlling the scene segmentation by allowing the user to merge similar scenes. The application also captures text on screen which is helpful in creating video descriptions. After the user has edited the descriptions, the descriptions are further converted from text to speech. The audio clips are then played along with the video. The project team conducted a user study with novice describers which proved that the application made the video description process much easier and less time-consuming. In the second phase of the User study with blind and visually impaired users, the AI-supported video descriptions were rated higher than the descriptions with no AI support. The application is currently being integrated with the YouDescribe website

6. FUTURE WORK

The project team is currently working on integrating the application with the YouDescribe website. A new User Interface for generating video descriptions with AI support is being developed using React. All the video description data created by the users of the YouDescribe website will be saved in the application's database. The user study system uses an RDBMS, PostgreSQL as a database whereas the YouDescribe app uses a no-SQL database, MongoDB. In the future, the PostgreSQL database will be migrated to MongoDB.

In the future, the team plans to build an interactive user interface for blind and visually impaired users where a user will be able to ask questions about the video based on visuals. For this, the application will be integrated with a machine learning model, visual dialogue. The user will activate the interactive mode by pressing a hotkey. On activating the interactive mode, the video will be paused, and the system will start recording the question. The question will be sent to the Visual dialogue model. The model will answer the question and the user will be able to hear and read the answer. The user will be asked to rate the answer to each question. All these questions will be used to train the Visual Dialogue model. The system developed for user study 2 will be used in future user studies.

7. REFERENCES

- [1] W3C Web Accessibility Initiative (WAI)
<https://www.w3.org/WAI/fundamentals/accessibility-intro/#important>
- [2] Beste F. Yuksel, Pooyan Fazli, Umang Mathur, Vaishali Bisht, Soo Jung Kim, Joshua Junhee Lee, Seung Jung Jin, Yue-Ting Siu, Joshua A Miele, and Ilmi Yoon
Increasing Video Accessibility for Visually Impaired Users with Human-in-the-Loop Machine Learning, Extended Abstracts of the 2020 CHI Conference on Human Factors in Computing Systems, April 25–30, 2020, Honolulu, HI, USA.
- [3] American Council of the Blind: <https://www.acb.org/adp/ad.html>
- [4] Ultimate Guide to Audio Description:
<https://www.3playmedia.com/resources/popular-topics/audio-description/>
- [5] 3PlayMedia Audio Description services:
<https://www.3playmedia.com/solutions/services/audio-description/>
- [6] LiveDescribe: <https://imdc.ca/completedprojects/livedescribe>
- [7] YouDescribe website: <https://youdescribe.org/>
- [8] <https://www.3playmedia.com/2017/06/28/when-and-when-not-to-add-audio-description-to-content/>
- [9] Branje, Carmen; Marshall, Susan; Tyndall, Ashley; and Fels, Deborah, "LiveDescribe" (2006) <http://aisel.aisnet.org/amcis2006/368>
- [10] 3Play Media: <https://www.3playmedia.com/>
- [11] 3Play Plugin: <https://www.3playmedia.com/solutions/features/plugins/3play-plugin/>
- [12] Microsoft Video Indexer:
<https://docs.microsoft.com/en-us/azure/media-services/video-indexer/>
- [13] Microsoft Azure Computer Vision documentation
<https://docs.microsoft.com/en-us/azure/cognitive-services/computer-vision/home>
- [14] IBM Code Model Asset Exchange: <https://github.com/IBM/MAX-Scene-Classifer>
- [15] Google Text-To-Speech API: <https://cloud.google.com/text-to-speech>
- [16] Google Cloud Vision AI <https://cloud.google.com/vision/>

- [17] Amazon Rekognition: <https://aws.amazon.com/rekognition/>
- [18] ListenByCode: <https://www.listenbycode.com/>
- [19] Amazon Relational Database Service: <https://aws.amazon.com/rds/>
- [20] BLEU: a method for automatic evaluation of machine translation
Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu IBM T. J. Watson
Research Center Yorktown Heights, NY 10598, USA
{papineni,roukos,toddward,weijing}@us.ibm.com
- [21] youtube-dl <https://github.com/yt-dl-org/youtube-dl/blob/master/README.md#readme>
- [22] youtube-dl Node.js <https://www.npmjs.com/package/youtube-dl>
- [23] FFMPEG <https://www.ffmpeg.org/>
- [24] Pythia: <https://github.com/facebookresearch/pythia>
- [25] IRB:
<https://research.oregonstate.edu/irb/frequently-asked-questions/what-institutional-review-board-irb>
- [26] SG Hart and others. 1988. Development of NASA-TLX: Results of empirical and theoretical research. In P. A. Hancock and N. Meshkati (eds.), Human Mental Workload (1988).
- [27] ARIA labels: <https://developer.mozilla.org/en>
- [28] Microsoft Azure Cognitive Services
https://azure.microsoft.com/en-us/services/cognitive-services/?ef_id=CjwKCAjwqdn1BRBREiwAEbZcRzzznJm7QtuPj5QCev-a1iEGDP0OuXeUzgnYARa5xFJ_mRHeCuihoC_tYQAvD_BwE:G:s&OCID=AID2000128_SEM_CjwKCAjwqdn1BRBREiwAEbZcRzzznJm7QtuPj5QCev-a1iEGDP0OuXeUzgnYARa5xFJ_mRHeCuihoC_tYQAvD_BwE:G:s&gclid=CjwKCAjwqdn1BRBREiwAEbZcRzzznJm7QtuPj5QCev-a1iEGDP0OuXeUzgnYARa5xFJ_mRHeCuihoC_tYQAvD_BwE